

Qurts: Automatic Quantum Uncomputation by Affine Types with Lifetime

KENGO HIRATA, University of Edinburgh, United Kingdom and Kyoto University, Japan
CHRIS HEUNEN, University of Edinburgh, United Kingdom

Uncomputation is a feature in quantum programming that allows the programmer to discard a value without losing quantum information, and that allows the compiler to reuse resources. Whereas quantum information has to be treated linearly by the type system, automatic uncomputation enables the programmer to treat it affinely to some extent. Automatic uncomputation requires a substructural type system between linear and affine, a subtlety that has only been captured by existing languages in an ad hoc way. We extend the Rust type system to the quantum setting to give a uniform framework for automatic uncomputation called Qurts (pronounced quartz). Specifically, we parameterise types by lifetimes, permitting them to be affine during their lifetime, while being restricted to linear use outside their lifetime. We also provide two operational semantics: one based on classical simulation, and one that does not depend on any specific uncomputation strategy.

CCS Concepts: • **Theory of computation** → **Quantum computation theory**; **Type structures**; **Operational semantics**.

Additional Key Words and Phrases: Quantum Programming, Rust, Uncomputation, Affine Type, Linear Type, Lifetime

ACM Reference Format:

Kengo Hirata and Chris Heunen. 2025. Qurts: Automatic Quantum Uncomputation by Affine Types with Lifetime. *Proc. ACM Program. Lang.* 9, POPL, Article 6 (January 2025), 28 pages. <https://doi.org/10.1145/3704842>

1 Introduction

Quantum information famously cannot be copied or discarded by the laws of nature. Consequently, programming languages for quantum computers typically enforce a *linear type discipline* to prevent misuse of quantum information [Altenkirch and Grattage 2005; Pratt 1992; Selinger and Valiron 2006; Wadler 1990]. The contraction rule, which allows copying, must be abandoned in order to obey the no-cloning theorem of quantum mechanics [Wootters and Zurek 1982]. Similarly, the weakening rule, which allows discarding, must be abandoned in order to prevent unintended implicit measurements [Kumar Pati and Braunstein 2000]. However, the latter restriction on weakening is cumbersome because it mandates that quantum values is explicitly transformed into classical data through measurement before dropping them.

Uncomputation is a technique in quantum programming that allows quantum values to be discarded without sacrificing quantum information. Discarding ‘dirty’ qubits has undesired effects on the rest of the computation via entanglement, so they first need to be ‘cleaned’. A qubit is cleaned by disentangling it from the other qubits and resetting it to its initial state. This *uncomputation* needs to be inserted into the computation at the appropriate place before discarding the qubit,

Authors’ Contact Information: Kengo Hirata, University of Edinburgh, Edinburgh, United Kingdom and Kyoto University, Kyoto, Japan, K.Hirata@sms.ed.ac.uk; Chris Heunen, University of Edinburgh, Edinburgh, United Kingdom, Chris.Heunen@ed.ac.uk.



This work is licensed under a Creative Commons Attribution 4.0 International License.

© 2025 Copyright held by the owner/author(s).

ACM 2475-1421/2025/1-ART6

<https://doi.org/10.1145/3704842>

giving a space-time trade-off [Meuli et al. 2019; Paradis et al. 2021, 2024; Venev et al. 2024]: it is more space-efficient to uncompute as early as possible, but more time-efficient to uncompute as late as possible. Moreover, uncomputing a qubit cannot be done too late – the process of uncomputation may use other qubits that are entangled with the qubit in question, and must be done before the information in those other qubits is lost. Thus, the timing of uncomputation is crucial.

Automatic uncomputation is a technique to insert uncomputation automatically in a program, so that the programmer does not need to worry about it. The idea was introduced by ReVerC [Amy et al. 2017] (in a non-linear setting), and gained further traction with the development the quantum programming language Silq [Bichsel et al. 2020], which implements *implicit automatic uncomputation* (alongside linear types). Silq’s ad hoc type system can statically verify that a function behaves like a classical Boolean circuit, which is a pivotal prerequisite if it is to be uncomputed. In that case the compiler automatically inserts the uncomputation without needing explicit annotation from the programmer. Consequently, Silq programmers can safely discard qubits implicitly; the type system assures that these qubits can be uncomputed, hence preventing the act of discarding from interfering with the rest of the computation. Silq has a purely functional core, but the full language has imperative features such as assignments, leading to puzzling behaviour, for example, inlining an assignment can cause type errors, or conversely, inlining can fix type errors and make type checks pass.

Automatic uncomputation can be thought of as a system that can type a qubit more loosely than linear type but more strongly than affine type: the qubit can be dropped when we have kept enough information to clean the qubit, but otherwise must be treated linearly. No existing language expresses such a subtle substructural type system, in which the type of a variable may be treated affinely only during certain periods in the computation.

Rust [Matsakis and Klock 2014; Rust Community 2024b] has a type system that tracks the *lifetime* of information. Explicit *ownership* of variables was introduced there to achieve memory safety and thread safety in the face of shared memory use. In Rust, while a shared reference, called *immutable reference*, with read-only permission is lent to another, write permission is suspended. When such references are no longer used, the owner of the resource can write again. The lifetime feature is used to ensure that the associated resource eventually ‘returns’ to its owner.

Qurts. This article introduces a Rust-like quantum programming language with a proper substructural type system that supports implicit automatic uncomputation, and that can statically verify the correctness of uncomputation. The type system has explicit lifetime and ownership of variables, like in Rust, and is meant to express values that can be uncomputed during their lifetime: in the type `#'a`, the lifetime annotation `'a` represents the interval during which the value can be treated affinely. Qurts supports a fragment of affine types in which variables can be dropped at any time: for example, a boolean value with type `#'static bool` can be treated affinely during the whole program since the lifetime is static, that is, the whole program. Qurts also has a fragment of linear types in which variables can never be dropped: for example, a qubit with type `#'0 qbit`, whose lifetime is empty, must be treated linearly.

Qurts also has immutable references `&'a` as in Rust. A variable of type `&'a qbit` is treated as a pointer to a qubit that can only be used for quantum control, that is, the qubit is treated as immutable data with respect to the computational basis. As in Rust, such immutable reference represents shared memory. Data of type `&'a qbit` can be freely copied or discarded, but doing so only copies the pointer to the qubit, not the qubit itself. The most important role of immutable references in Qurts is not to represent shared data, but to represent quantum information that must be kept alive for some time `'a` to uncompute other qubits which are entangled with the qubits that the references point to.

Table 1. Some quantum programming languages with uncomputation types

	ReVerC	ReQWIRE	Silq	Qrisp	Qurts
Automatic uncomputation	✓		✓	✓	✓
Linear types		✓	✓	✓	✓
Affine types			*1		✓
Immutable borrow			✓*2		✓
Lifetime					✓
Flexible uncomputation strategy					✓

Semantics. We provide two semantics for Qurts, that we prove equivalent. Both are small-step operational semantics, but each handles uncomputation differently.

Simulation semantics interprets dropping variables by simply discarding the qubits according to the mathematical definition of uncomputation, which converts a state $\sum_{i \in \{0,1\}^n} |\varphi_i\rangle |i\rangle$ of $m + n$ qubits into the state $\sum_{i \in \{0,1\}^n} |\varphi_i\rangle |0 \dots 0\rangle$, forcing the last n qubits into the state $|0\rangle$. This operation can easily be simulated on a classical computer, but cannot be executed as a quantum circuit because it is not reversible and need not preserve probabilities. However, we prove that the Qurts type system guarantees that this semantics is reversible and preserves probabilities.

Uncomputation semantics interprets Qurts programs as *reversible pebble games* [Bennett 1989]. These form models of quantum uncomputation [Bhattacharjee et al. 2019; Meuli et al. 2019]: a quantum circuit without uncomputation can be interpreted as a game, and any winning strategy of the game corresponds to a circuit with uncomputation inserted. We show that Qurts programs can be interpreted as an extended version of pebble games. This gives a more flexible semantics, that allows us to include more efficient uncomputation strategies than simulation semantics. As far as we know, Qurts is the first quantum programming language that integrates reversible pebble games for flexible uncomputation strategies.

Contributions. After discussing background in Section 2, we present the following contributions.

- Section 3 introduces the quantum programming language Qurts with many examples. It successfully represents automatic uncomputation by simply annotating a lifetime to the types in Rust.
- Section 4 presents operational semantics based on classical simulation, which is not generally physically implementable. We prove that the Qurts type system guarantees that the semantics of dropping variables is reversible and preserves probability.
- Section 5 provides another operational semantics, based on reversible pebble games, that is non-deterministic and gives physical implementation. We prove that the simulation and uncomputation semantics are equivalent.

Section 6 discusses related work, briefly summarised in Table 1. Finally, Section 7 lists directions for further investigation.

2 Background

This section summarises the basic notions that this article builds on, namely quantum computation, the Rust type system, and pebble games.

¹Silq has a `qfree` annotation for functions, but not for types.

²Silq has `const` types, but they do not follow move semantics.

2.1 Quantum Computation

Let us start with a brief overview of the essential concepts in quantum computing. For more details we refer to textbooks such as [Nielsen and Chuang 2010; Yanofsky and Mannucci 2008].

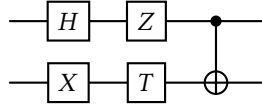
Qubits. The basic carrier of quantum information is the *qubit*. The state of a qubit is a unit vector $\varphi = \begin{pmatrix} x \\ y \end{pmatrix}$ in the Hilbert space $\mathbb{C}^2$, that is, a pair of complex numbers x and y such that $|x|^2 + |y|^2 = 1$. This vector is often written in *ket notation* $|\varphi\rangle$. Two special vectors are the *computational basis* states $|0\rangle = \begin{pmatrix} 1 \\ 0 \end{pmatrix}$ and $|1\rangle = \begin{pmatrix} 0 \\ 1 \end{pmatrix}$; a general state $|\varphi\rangle$ is in a *superposition* of these two. Two such states that we will often use are $|+\rangle = \frac{1}{\sqrt{2}}|0\rangle + \frac{1}{\sqrt{2}}|1\rangle$ and $|-\rangle = \frac{1}{\sqrt{2}}|0\rangle - \frac{1}{\sqrt{2}}|1\rangle$.

Entanglement. In a system with multiple qubits, the state is a unit vector in the *tensor product*. For example, if the first qubit is in state $|0\rangle$, and the second qubit is in state $|1\rangle$, then the state of the compound system is the vector $|0\rangle \otimes |1\rangle \in \mathbb{C}^2 \otimes \mathbb{C}^2$, also written as $|01\rangle$. Not all states of a system with multiple qubits are of this form. For example, the state $\frac{1}{\sqrt{2}}(|00\rangle + |11\rangle)$ is *entangled*: it cannot be written in the form $|\varphi\rangle \otimes |\psi\rangle$.

Circuits. Qubits are acted upon by unitary matrices, also called *quantum gates*. Standard one- and two-qubit gates include

$$H = \frac{1}{\sqrt{2}} \begin{pmatrix} 1 & 1 \\ 1 & -1 \end{pmatrix} \quad X = \begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix} \quad Z = \begin{pmatrix} 1 & 0 \\ 0 & -1 \end{pmatrix} \quad T = \begin{pmatrix} 1 & 0 \\ 0 & \frac{1+i}{\sqrt{2}} \end{pmatrix} \quad CX = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}.$$

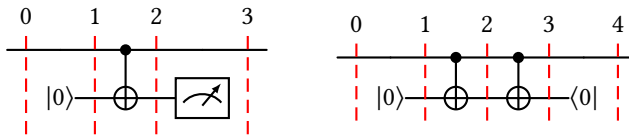
The gate H is called the *Hadamard gate*. A quantum computation is described by a *quantum circuit*, which is a product of tensor products of quantum gates. For example, here is the two-qubit quantum circuit notation for the 4-by-4 unitary matrix $CX \circ (Z \otimes T) \circ (H \otimes X)$:



Measurement. The outcome produced by a quantum computation is read out by a *measurement*. This is a probabilistic procedure, that, when given the state $x|0\rangle + y|1\rangle$ of a qubit, returns outcome 0 with probability $|x|^2$, and outcome 1 with probability $|y|^2$. Measurement collapses superposition, and afterwards the state is $|0\rangle$ or $|1\rangle$, respectively, depending on the outcome. In quantum circuits, measurements are drawn as $\boxed{\text{meter symbol}}$.

Quantum circuits can have *auxiliary qubits*, that are not part of the input or output, but only store some temporary information. Without loss of generality, they are first initialised in state $|0\rangle$. One way to discard an auxiliary qubit is to measure it at the end of its usage. When we can guarantee that an auxiliary qubit is in state $|\varphi\rangle$ at the point of measurement, we may also draw $\langle\varphi|$ instead of $\boxed{\text{meter symbol}}$. This *bra* notation turns the vector $|\varphi\rangle = \begin{pmatrix} x \\ y \end{pmatrix} \in \mathbb{C}^2$ into its adjoint $\langle\varphi| = (x^* \ y^*) : \mathbb{C}^2 \rightarrow \mathbb{C}$.

Uncomputation. Measuring one qubit in a multiple-qubit system can affect the whole state. Consider the following left quantum circuit that operates on a single qubit, using one auxiliary qubit.



The top qubit is only used as a control, and hence one might expect the output to equal the input. However, the mere fact that the auxiliary qubit is initialised, interacts with the data qubit, and is

discarded alters this expected behaviour. Suppose that the input at step 0 is $|+\rangle = \frac{1}{\sqrt{2}}(|0\rangle + |1\rangle)$. This evolves to $\frac{1}{\sqrt{2}}(|00\rangle + |10\rangle)$ at step 1, and to $\frac{1}{\sqrt{2}}(|00\rangle + |11\rangle)$ at step 2. But after the measurement, at step 3, the output is $|0\rangle$ with probability $\frac{1}{2}$, and $|1\rangle$ with probability $\frac{1}{2}$. This behaviour occurred because the auxiliary qubit was discarded while it was in a *dirty* state, that is, it was not guaranteed to be in the same state it was initialised in. To prevent this behaviour, the auxiliary qubit needs to be cleaned before discarding it. We can do this by inserting an *uncomputation* before discarding as in the right circuit above. The inserted circuit between steps 2 and 3 uncomputes everything that happened to the auxiliary qubit, guaranteeing that it is measured in the same state it was initialised in, and therefore has no effect on the rest of the computation. Indeed, if the input state at step 0 is $|+\rangle$, then at step 3 the state is now $\frac{1}{\sqrt{2}}(|00\rangle + |10\rangle)$, and so the output at step 4 is again $|+\rangle$, as expected. The general definition is as follows.

Definition 2.1. Given a quantum state $|\varphi\rangle = \sum_{i \in \{0,1\}^n} |\varphi_i\rangle|i\rangle$ of $m + n$ qubits, an *uncomputation* on the last n qubits is an operation that maps $|\varphi\rangle$ to $\sum_{i \in \{0,1\}^n} |\varphi_i\rangle|0 \dots 0\rangle$.

This definition differs slightly from correctness of uncomputation as defined in [Paradis et al. 2021, 2024]. There, correctness is defined as a predicate on a pair of operations $(G, \mathcal{G})$ stating that $\mathcal{G}$ is an operation that does the same as G but leaves auxiliary qubits uncomputed. Our definition above focuses on the state of the qubits after the operation rather than the operation itself. This may seem less general because uncomputation is defined only as a post-processing step, but this is not a problem because this paper allows uncomputation at any point of the computation.

The following inequality shows that the norm of uncomputed state is always larger than or equal to that of the original state, with equality holds if and only if $\{|\varphi_i\rangle\}_i$ are orthogonal.

$$|\varphi\rangle^2 = \left| \sum_{i \in \{0,1\}^n} |\varphi_i\rangle|i\rangle \right|^2 = \sum_{i \in \{0,1\}^n} \left| |\varphi_i\rangle \right|^2 \leq \left| \sum_{i \in \{0,1\}^n} |\varphi_i\rangle \right|^2 = \left| \sum_{i \in \{0,1\}^n} |\varphi_i\rangle|0 \dots 0\rangle \right|^2$$

This means that the total probability of measurement success before uncomputation is no less than after. As the total probability cannot be increased by any physical operation, uncomputation is physically realisable only if the set of states $\{|\varphi_i\rangle\}_i$ are orthogonal, which is not always the case.

Uncomputation can also be used for space optimisation, as it allows auxiliary qubits to be ‘deallocated’ in order to save space. However, uncomputation may cost time because an uncomputed value might need to be recomputed later. Vice versa, leaving a value uncomputed for longer occupies memory space, but may save computation time when the value is reused later. We will see this trade-off between time and space later in Section 2.3.

Quantum Control. The quantum state $|0\rangle$ is often identified with the boolean value `false`, and $|1\rangle$ with `true`. Quantum circuits can implement boolean operations this way. For example, the X gate implements the not operation. When a unitary function U maps a state $|x\rangle \in \mathbb{C}^2 \otimes \dots \otimes \mathbb{C}^2$ to $|f(x)\rangle$ for a bijective function $f: \{\text{false}, \text{true}\}^n \rightarrow \{\text{false}, \text{true}\}^n$, we call U the *lift* of f . Together with a function that introduces a new qubit $|0\rangle: \mathbb{C} \rightarrow \mathbb{C}^2; 1 \mapsto |0\rangle$, this definition of lift can be extended to any injective function $f: \{\text{false}, \text{true}\}^n \rightarrow \{\text{false}, \text{true}\}^m$.

There are quantum analogues of conditional operations in classical computing, called *quantum control* [Sabry et al. 2018; Ying 2016]. Some programming languages allow quantum control flow similar to classical control flow [Altenkirch and Grattage 2005; Bichsel et al. 2020]. For example, we denote the quantum analogue of `if` statements by the command `qif x { U(y) } else { V(y) }`. It can be implemented with *controlled gates*. Given any n -qubit gate U , the controlled- U operation is an $n + 1$ -qubit unitary that maps $|0\rangle|y\rangle \in \mathbb{C}^2 \otimes \mathbb{C}^{2^n}$ to $|0\rangle|y\rangle$ and $|1\rangle|y\rangle$ to $|1\rangle(U|y\rangle)$. For example, the Controlled- X gate is the matrix CX from above. This controlled- U operation represents the then branch of the `qif` statement. To implement the else branch, we need negatively-controlled- V

operations, which conjugates Controlled-V with X gates: $X(x)$; $\text{qif } x \{ V(y) \}; X(x)$; . Since the *control* qubit x can be in superposition, both branches are executed in superposition, which is a key difference from classical control flow, where only one branch is executed. Gates can similarly be controlled on multiple qubits; a *single target gate* is a n -qubit quantum gate that can be represented as the following map for a boolean function $i: \{x_1, \dots, x_n\} \rightarrow \{0, 1\}$ where $\neg^1 x = \neg x$ and $\neg^0 x = x$.

$$|x_1 \dots x_n\rangle|y\rangle \mapsto |x_1 \dots x_n\rangle|y \oplus (\neg^{i_{x_1}} x_1 \wedge \dots \wedge \neg^{i_{x_n}} x_n)\rangle$$

For example, the gate $|x_1 x_2\rangle|y\rangle \mapsto |x_1 x_2\rangle|y \oplus (x_1 \wedge x_2)\rangle$ is called the *Toffoli gate*.

2.2 Rust: Lifetime, Borrowing, and Ownership

The Rust programming language [Matsakis and Klock 2014; Rust Community 2024b] enforces memory safety via a type system, and has a high level of performance optimisation. We briefly overview aspects we adapt.

Ownership: Move versus Copy. Instead of garbage collection, Rust uses a system of *ownership* [Rust Community 2024d]. In this system, each value has a single *owner* at a time, and as soon as the owner goes out of scope, the value is dropped. This ownership is *moved* when the value is assigned to another variable, or passed to a function. After the ownership is moved to another variable, the original variable loses access to the value. For example, in the code snippet below, the variable `s1` owns a string "hello" after running line 1. When `s1` is assigned to `s2` in line 2, the ownership of the string *moves* from `s1` to `s2`. Since Rust assignments are not copies but moves, the value of `s1` is no longer accessible after line 2. Thus, the `print!` call with `s1` in line 3 will not compile.

```
1 let s1 = String::from("hello");
2 let s2 = s1;
3 print!("{s1}, world!");
```

Ownership is underlain by *affine types*, substructural types that prohibit copying of values. This type discipline enables Rust to detect when a value becomes orphaned and free its associated memory. Moreover, by not regarding assignments as copies, Rust can avoid unnecessary deep copying of values, which can be expensive in terms of time and memory.

References. Rust's ownership system prohibits a value from having multiple owners at the same time. However, sometimes it is useful to share a value using a pointer. Rust allows this by using *references*. [Rust Community 2024a]. A reference is essentially a pointer to a value, and does not take ownership of the value. Instead, it *borrow*s the value from the owner. There are two types of references in Rust: *immutable references* and *mutable references*. Immutable references allow the value to be read but not modified, while mutable references allow the value to be modified. When a variable is immutably borrowed, there can be multiple references to the value, but the value cannot be modified through the reference. On the other hand, when a variable is mutably borrowed, there can be only one reference to the value, and the value can only be read or updated through the reference. After all references go out of scope, the owner regains full access to the value. In the code snippet below left, since the value which `x` owned in line 2 moved to `y` in line 3, the value of `x` is no longer accessible in line 4. However, in the code snippet on the right-hand side, the value of `x` is immutably borrowed in line 3, so the value that `x` owns is still readable in line 4.

```
1 fn main() {
2   let x = [1,2,3];
3   let y = f(x); // value of x is moved
4   print!("{x},{y}"); // so x lost access
5 }
6 fn f<T>(x : T) -> T {
7   x
8 }
```

```
1 fn main() {
2   let x = [1,2,3];
3   let y = f(&x); // x is borrowed
4   print!("{x},{y}"); // x can be read
5 }
6 fn f<T>(x : T) -> T {
7   x
8 }
```


Lifetime. To know when a reference is no longer needed, and the owner can regain full access to the value, Rust uses the concept of *lifetime* [Rust Community 2024c]. Every reference in Rust has a lifetime: the scope during which that reference is valid. This prevents dangling pointers or race conditions by ensuring that when someone has write access to a value, no one else has any access to the value. For example, the following right Rust code will not compile: The variable `r` has a lifetime of lines 1–6, and `x` has a lifetime of lines 3–5.

Storing a reference to `x` in `r` in line 4 is not allowed, because the lifetime of `x` ends before that of `r`. Most of the time lifetimes are implicit and inferred, just like most of the time types are inferred. However, when you want to define a function whose outputs include references, you need to explicitly annotate the lifetimes. This is because, if a reference is returned as a function output, it becomes difficult to track the aliveness of the reference statically. For example, `fn f<'a>(x: &'a bool)-> &'a bool` declares that the function `f` takes a reference to a boolean value with (external) lifetime `'a`, and returns a reference to a boolean value with the same lifetime.

```

1 let r;
2 {
3     let x = 5;
4     r = &x;
5 }
6 print!(r);

```

Similarity to Quantum Uncomputation. Uncomputation and garbage collection are similar in that both manage memory, and both trade-off time against space. Automatic uncomputation and Rust share the feature that the type system gives the compiler hints, and the compiler manages memory, so the programmer does not have to think about it.

Quantum computation has adopted linear type systems because generic quantum states cannot be copied or discarded. There is a quantum analogue of copying a quantum state defined by $\mathbb{C}^2 \ni |i\rangle \mapsto |ii\rangle \in \mathbb{C}^2 \otimes \mathbb{C}^2$, which respects the computational basis $\{|0\rangle, |1\rangle\}$. However, implicitly performing this copy-like operation is problematic in quantum computation because it entangles the original state with the copy, which interferes with the computation. Also, this operation uses a two-qubit gate, which is expensive in terms of error rates.

2.3 Reversible Pebble Games

Reversible pebble games were first applied to quantum uncomputation in [Meuli et al. 2019]. To understand them, consider the example program ‘`and(nand(not(and(a,b)),not(c)),d)`’. It is equivalent to the following pseudocode computing `w` by uncomputing temporary `x, y, z, v`:

```
x := And(a,b); y := Not(x); z := Not(c); v := Nand(y,z); w := And(v,d); uncomp(x,y,z,v)
```

From the program, we construct the *circuit graph* at the top left in Figure 1: a directed graph whose vertices correspond to gates, and whose edges represent dependencies between gates. For example, the edge from `x` to `y` means that `y` uses the result of `x` as its input.

A reversible pebble game is played on a directed acyclic graph, by placing tokens called pebbles on vertices, according to the following rules:

- You can place a pebble on a vertex when all its predecessors are already pebbled.
- You may remove a pebble from a vertex when all its predecessors are pebbled.

Intuitively, the vertices represent a value during program execution, and a pebble indicates that this value is stored in the memory. The aim of the game is to get pebbles on all vertices except those that are uncomputed. Typically, this means you have to get a single pebble on the last vertex, with no other vertices being pebbled. In our example, the goal is to get a pebble only on `w`.

This case allows two pebbling strategies. The first, naive, strategy puts pebbles on `x, y, z, v`, and `w` in this order, and then removes all pebbles but the one on `w` again in the reverse order. This strategy uses 5 pebbles and takes 9 execution steps. Another strategy is illustrated in Figure 1. The

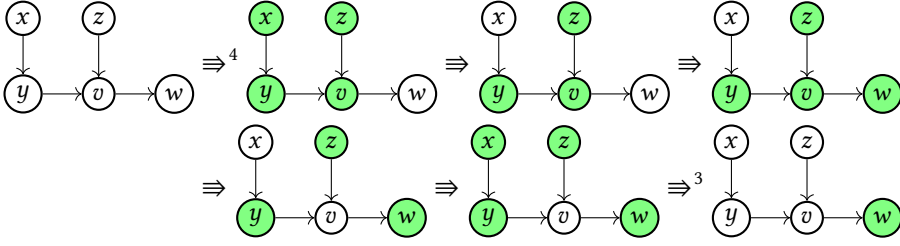


Fig. 1. An example strategy with a minimum number of pebbles

first 4 steps are the same as the naive strategy, but then it removes the pebble on x . This strategy uses only 4 pebbles at the same time, but takes 11 time steps.

Each pebbling strategy corresponds to a quantum circuit. A pebble represents a qubit, and a vertex represents a quantum state. When a pebble is on a vertex, it indicates that the qubit is in the state of that vertex. Placing a pebble on a vertex corresponds to storing the qubit's state in memory, while removing a pebble corresponds to uncomputing the qubit. Thus, the number of steps in the pebbling strategy corresponds to the number of gates in the quantum circuit. For example, the step putting a pebble on y in the configuration $(x) \rightarrow (y)$ corresponds to computing $|x\rangle|0\rangle \mapsto |x\rangle|\neg x\rangle$, since y is the negation of x in the example program. As seen in the example, the pebbling strategy affects the number of qubits and gates of the eventual quantum circuit. This gives a trade-off between time and space, whose optimisation is PSPACE-complete [Chan et al. 2015].

3 Qurts

This section introduces Qurts³⁴. Section 3.1 first overviews its features and capabilities by way of example. Section 3.2 then more formally introduces the syntax and type system.

3.1 Qurts by Example

Every variable in Qurts has a type, the primitive types being `bool` and `qbit`. Declaring a variable with explicit type requires stating ownership, as in Rust: either the variable owns its value, as in `x: #qbit`, or the variable immutably borrows its value, as in `x: &qbit`, or the variable mutably borrows its value, as in `x: &mut qbit`. Functions like `H` are applied to variables like `x: qbit` using the command `x.H()`; or the expression `H(x)`.

Grover's algorithm. We begin with a real-world example of Qurts: Grover's algorithm [Grover 1996]. The following very succinct code snippets implement the inner loop of the algorithm, illustrating Qurts' core functionality of automatic uncomputation.

```
qif &oracle(&x1) { phase( $\pi$ ) }
```

```
qif &non_zero(&x1) { phase( $\pi$ ) }
```

The fragments above are syntactic sugar for their equivalents below with explicit uncomputation.

```
let r = &x1;
let tmp: #qbit = oracle(r);
qif &tmp { phase( $\pi$ ) } else { noop };
drop tmp;
```

```
let r = &x1;
let tmp: #qbit = non_zero(r);
qif &tmp { phase( $\pi$ ) } else { noop };
drop tmp;
```

³⁴We follow the precedent of naming programming languages after precious materials, such as Ruby. Just like Perl abbreviates 'pearl', Qurts abbreviates 'quartz'. Additionally, Qurts is an anagram of 'Q' and 'Rust'.

⁴We will use some standard features from Rust in this article that are not formally defined in Section 3.2 below, but that are part of an implementation of Qurts in development.


```

fn grover(
  x:&mut qbit,
  y:&mut qbit,
  z:&mut qbit
) {
  x.H(); y.H(); z.H();
  for _ in 0..2 {
    qif &oracle(&x, &y, &z) { phase( $\pi$ ) }
    grover_diffusion(x, y, z)
  }
}

fn grover_diffusion(
  x:&mut qbit,
  y:&mut qbit,
  z:&mut qbit
) {
  x.H(); y.H(); z.H();
  qif &non_zero(&x, &y, &z) { phase( $\pi$ ) }
  x.H(); y.H(); z.H();
}

```

Fig. 2. Grover's algorithm for 3 qubits in Qurts.

The function call `phase( $\pi$ )` represents the map $-1: \mathbb{C} \rightarrow \mathbb{C}$ that multiplies the quantum state by -1 . Calling it within the `qif` causes the Z gate to be applied to the qubit owned by `tmp`. The `drop` statement at the end uncomputes the temporary qubit. This uncomputation is possible because `tmp` is obtained by the function call `oracle(r)` or `non_zero(r)`, and since the value which `r` refers to is not used after the function call, we can apply the reverse operation of the function call to `r` and `tmp` to uncompute the temporary qubit. The full algorithm is listed in Figure 2 (for 3-qubit input to keep it explicit and not import Rust syntax for arrays). See [Hirata and Heunen 2024, Appendix A] for more details.

From the type system perspective, the functions `oracle` and `non_zero` have the same signature `fn (&qbit, &qbit, &qbit) -> #qbit`. This type means that the function takes immutable references to qubits and returns a qubit.

Implementing Boolean circuits. Next, we implement in Qurts boolean circuits, whose return qubit can be uncomputed, such as the function `non_zero` in the previous example. The Qurts code on the left below implements a binary AND on qubits.

```

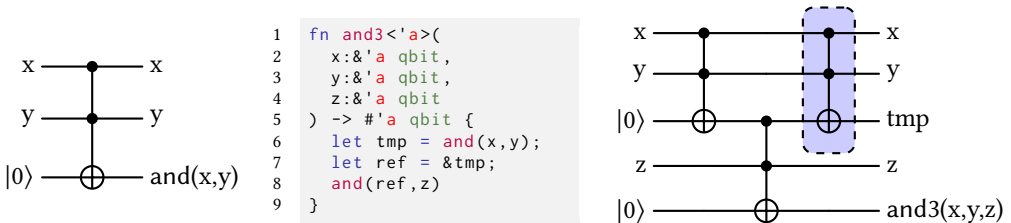
fn and<'a>(x:&'a qbit, y:&'a qbit) -> #'a qbit {
  qif x {
    qif y { |1> } else { |0> }
  } else {
    |0>
  }
}

fn and3<'a>(
  x:&'a qbit,
  y:&'a qbit,
  z:&'a qbit
) -> #'a qbit {
  and(&and(x, y), z)
}

```

Note that after calling this function `and`, the caller can use three qubits: the two input qubits and the result qubit. The input variables `x` and `y` are immutable references, so they are not consumed by the function `and`. On the other hand, the result is of type `#qbit` so will be moved to the caller.

The code on the right above uses the 2-qubit AND to build the 3-qubit version `and3`. Notice how the first argument of the outer call to the 2-qubit AND has to be converted from `and(x, y):#qbit` to `&and(x, y):&qbit`. The function `and` gets compiled to the circuit on the left below.



Let us unfold the original function `and3` to make the auxiliary qubit `tmp` explicit. This gets compiled to the first two Toffoli gates in the circuit on the right above – before any uncomputation: the auxiliary qubit `tmp` still needs to be uncomputed before it can be discarded. One way to uncompute

the auxiliary qubit is to apply the third Toffoli gate, marked in the circuit. If the input was $|ij0k0\rangle$, the state after the first two Toffoli gates is $|ij(i \wedge j)k(i \wedge j \wedge k)\rangle$, and after the uncomputation it becomes $|ij0k(i \wedge j \wedge k)\rangle$. Similarly, we can build a 3-(qu)bit OR function to implement `non_zero`.

Lifetime annotations. We did not go into detail above, but `and` and `and3` required lifetime annotations `'a`. As in Rust, all references in Qurts have lifetimes. In addition, Qurts also has lifetime annotation of linearity `#'a T`. This type contains values that can only be dropped/uncomputed by the end of the lifetime `'a`. For example, the signature `fn and<'a>(x:&'a qbit, y:&'a qbit)->'a qbit` means that the function `and` takes two immutable references to qubits and returns a qubit which can only be uncomputed as long as the references are available.

The key typing rule in Qurts is the lifetime annotation for the return value of `qif` statements.

```
// r: &'a qbit
let x: #'a T = qif r { M } else { N }; // M, N: T
```

This illustrates that, if the control qubit `r` has type `&'a qbit`, the return value of the `qif` statement has type `#'a T` with the same lifetime `'a`, where `T` is the type of the branches. In other words, it restricts the return value to be uncomputed only until the end of `'a`. The reason for this restriction is that the control is required to uncompute the return value, but the control is only available until the lifetime `'a` ends.

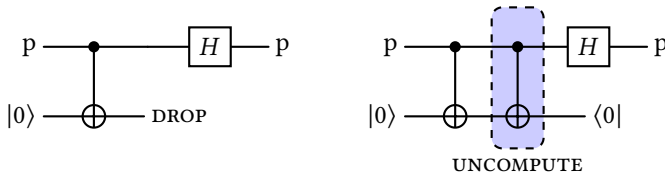
Lifted functions. In Qurts, you can *lift* any classical injection $c: \{\text{true}, \text{false}\}^n \rightarrow \{\text{true}, \text{false}\}^m$ to a primitive function `[c]: #'a [qbit;n] -> #'a [qbit;m]` which takes a list of qubits and returns a list of qubits. For example, `[not]` is a 1-qubit lifted function which represents the X-gate, `[cnot]` is a 2-qubit lifted function which represents the controlled-X gate, and `[swap]` is a 2-qubit lifted function which represents the SWAP gate. These can be used to make affine qubits from other affine qubits without borrowing them. The lifetime annotation indicates that the output is uncomputable for the same period of time that the input was uncomputable.

Uncomputability via types. The following code explains how lifetimes assigned to linearity can detect cases where uncomputation is not trivially possible through type checking.

```
1 {
2   // mut p: #qbit
3   let q = |0>;
4   let q = qif &p { [not](q) } else { q };
5
6   p.H();
7   return p;
8 } // q is dropped before here
```

```
1 {
2   // mut p: #qbit
3   let q = |0>;
4   let q: #'a qbit =
5     qif &'a p { [not](q) } else { q };
6   drop q; endlft 'a;
7   p.H();
8   return p;
9 }
```

Both snippets are valid Qurts, and are equivalent; the code on the right has merely inserted inferred lifetime. Line 6 on the right is the explicit drop of the qubit `q` and explicit declaration of the end of lifetime `'a`. This code compiles to the quantum circuit on the left below – before the uncomputation of the auxiliary qubit `q`. The circuit on the right below is one of the possible circuits with uncomputation inserted for the auxiliary qubit `q`.



On the other hand, the following code causes a type error.

```

1 { // mut p: #qbit
2   let q = |0>;
3   let q = qif &p { [not](q) } else { q };
4
5   let p = qif &q { [not](p) } else { p };
6
7   return p;
8 } // q is dropped before here

```

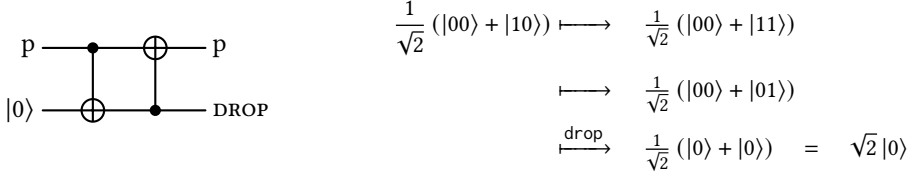
```

1 { // mut p: #qbit
2   let q = |0>;
3   let q: #'a qbit =
4     qif &'a p { [not](q) } else { q };
5   endlft 'a;
6   let p =
7     qif &'b q { [not](p) } else { p };
8   endlft 'b;
9   return p;
10 } // Type error: q cannot be dropped

```

This program replaces the Hadamard gate from before with another `qif` statement. It uses two different lifetimes `'a` and `'b`, where `'a` ends at line 5. This lifetime end is necessary: all references to the qubit `p` must have expired before the `[not]` in line 7 can mutably use it. In other words, the qubit `p` borrowed in line 4 must be taken back before line 7. Here, we can see that the type of variable `q` is bounded by `'a` at line 3. This is because of the typing rule for `qif` statement: the return value of the `qif` statement has the lifetime used in the control qubit. Therefore, the type of the variable `q` becomes linear after the end of `'a` at line 5. We have to drop the qubit `q` in this program. But `q` is linear after line 5, and since it is last used in line 7, it cannot be dropped. This program thus causes a type error.

In fact, this type error detects when uncomputation is impossible. This computation can be represented as the following quantum circuit. If qubit `p` starts in state $|+\rangle$, then the computation proceeds as on the right below, where the first qubit is `p` and the second qubit is `q`. The probability of the output state is $2 = (\sqrt{2})^2$, which exceeds 1, showing that qubit `q` is not uncomputable.

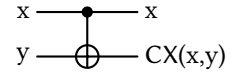


Two implementations of CX. The CX gate, in quantum circuit form as on the right below, can be written in two ways in Qurts. One uses `qif` and `[not]` as on the left below.

```

1 fn my_cnot<'a>(x: #'a qbit, y: #'a qbit) -> #'a qbit {
2   let y' = qif x { [not](y) } else { y }; return y'
3 }

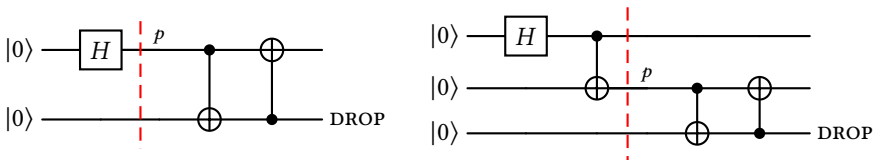
```



The other implementation is given by a lifted function – the lift of the classical boolean controlled-NOT gate. This can be written as the primitive Qurts-term below.

```
[cnot] : ( #'a qbit, #'a qbit ) -> ( #'a qbit, #'a qbit )
```

The type difference may seem minor, but it is important to verify uncomputability. We have already seen that the second qubit in the circuit on the left below is not uncomputable, and that this circuit can in fact not result from compilation of Qurts code using `qif` because of a type error. Using a lifted function `[cnot]` also causes a type error. Firstly, the output of the lifted function has the same type as its input. Therefore, if we can drop the second qubit, then we must be able to drop the first qubit at `p`. But since the quantum state of the first qubit was $|+\rangle$ at `p`, this could not be affine and hence is not uncomputable.



On the other hand, the circuit on the right above is valid, and the third qubit is uncomputable. Indeed, the final quantum state of the circuit on the right above is $\frac{1}{\sqrt{2}}(|000\rangle + |101\rangle)$, and after uncomputing the third qubit, the final quantum state is $\frac{1}{\sqrt{2}}(|00\rangle + |10\rangle)$. However, this can not be generated by the Qurts program using CX gates implemented with `qif` for the same reason as above: we cannot mutably use the second qubit in the last CX gate. On the other hand, the Qurts program using the lifted function `[cnot]` can generate this circuit, because at p the second qubit can be affine, as in line 2 of the following code.

```
1 let x:&qbit = &|0>.H();
2 let p:#qbit = qif x { |1> } else { |0> };
3 let q:#qbit = |0>;
4 let (p,q) = [cnot](p, q);
5 let (q,p) = [cnot](q, p);
6 drop q;
```

Dropping variables. In Qurts, you can define a function which drops a qubit even if the function itself does not know how to uncompute it. The code on the left below implements a function `forget` which drops the qubit `x` after lifetime `'a`. The constraint `<'a!='0>` in the signature means that the lifetime `'a` is still valid when the function is called. This verifies that the qubit `x` is dropped while it is affine. The code on the right below implements a function `reinitialise` which implicitly drops the qubit `x` inside the `qif` statement.

```
1 fn forget<'a!='0>(x: #'a qbit) {
2   // x is affine during the lifetime 'a
3 }
4
5 // the code above is equivalent to
6 fn forget<'a!='0>(x: #'a qbit) {
7   drop x;
8 }
```

```
1 fn reinitialise<'a!='0>(
2   x: #'a qbit,
3   y: &'a qbit
4 ) -> #'a qbit {
5   qif y { // drop x;
6     |0>
7   } else {
8     x
9   }
10 }
```

Compare this with the identity function `fn id(x: #'a qbit) -> #'a qbit { x }`, which needs no restrictions on the lifetime `'a`.

Toy example. To illustrate semantics we will use a final toy example.

```
1 let mut x: #'0 qbit = |0>;
2 x.H();
3 let r = &x;
4 let y = qif r { |1> }
5           else { |0> };
6 meas(x)
```

```
1 let x0 = [0]();
2 x0 as #+qbit;
3 let x1 = H(x0);
4 newlft α;
5 let r = &αx1;
6 let y = qif r { let z = [1](); z }
7           else { let z = [0](); z };
8 drop r;
9 drop y;
10 endlft α;
11 let ret = meas(x1);
12 ret
```

The code on the left is a simple quantum program in Qurts. The only feature we have not discussed yet is the `meas` function, which measures a qubit and returns a bit. On the right is the same program in the core language which we will define formally in the next section. Let us explain it briefly. All the initialization of a qubit are now identified with the call of the lifted function `[0]` or `[1]` where `0` is the constant function of false, and `1` is that of true. For simplicity, we omit mutable variables. Line 2 coerces the variable `x0` to `#+qbit`, meaning that it will be treated linearly. We make every introduction and elimination of lifetime explicit with `newlft` and `endlft`. This is the same for `drop`.

Syntax:

<i>Program</i>	Π	$::=$	$F_0 \dots F_{n-1}$
<i>Function</i>	F	$::=$	$\text{fn } f \Sigma B$
<i>Signature</i>	Σ	$::=$	$A (x_0 : T_0, \dots, x_{n-1} : T_{n-1}) \rightarrow T$
<i>Block</i>	B	$::=$	$\{ S; x \}$
<i>Statement</i>	S	$::=$	$\text{noop} \mid S_1; S_2 \mid \text{newlft } \alpha \mid \text{endlft } \alpha \mid \alpha \leq \beta \mid x \text{ as } T$ $\mid \text{let } y = \&^\alpha x \mid \text{let } y = e \mid \text{let } (y_0, y_1) = x \mid \text{drop } x$
<i>Expression</i>	e	$::=$	$x \mid \text{false} \mid \text{true} \mid () \mid (x_0, x_1) \mid \text{copy } x$ $\mid \text{meas}(x) \mid U(x) \mid [c](x) \mid f(\alpha_0, \dots, \alpha_{m-1})(x_0, \dots, x_{n-1})$ $\mid \text{if } x B_t \text{ else } B_f \mid \text{qif } x B_{ 1\rangle} \text{ else } B_{ 0\rangle}$
<i>Lifetime Preorder</i>	A	$::=$	$\langle A, R \rangle$ where A : a set of lifetime variables, R : a preorder on $A \cup \{\perp, \top\}$ $(x, y \text{ are local variables, } \alpha, \beta \text{ are lifetime variables, } f \text{ is a function name.})$

Fig. 3. Qurts syntax

Types:

<i>Lifetime</i>	α	$::=$	$\alpha \mid \perp \mid \top$
<i>Type</i>	T	$::=$	$\text{bool} \mid \text{qbit} \mid ()$ $\mid T_1 \times T_2 \mid \&^\alpha T \mid \#^\alpha T$

Contexts:

<i>Type Context</i>	Γ	$::=$	$x_0 : {}^{a_0}T_0, \dots, x_{n-1} : {}^{a_{n-1}}T_{n-1}$
<i>Aliveness</i>	$\mathbf{a}$	$::=$	$\text{active} \mid \dagger \alpha$

Fig. 4. Qurts types and judgement contexts

3.2 Syntax and Type System

We formalize the core language of Qurts by presenting its syntax and type system. We only overview them in this section. The detailed formalisation is in [Hirata and Heunen 2024, Appendix B].

3.2.1 Syntax.

Program. The syntax of Qurts-core, which is the core language of Qurts, is defined in Figure 3. Most of the syntax and typing rules are taken from Calculus of Ownership and Reference [Matsushita et al. 2020], a formalised core language of Rust. A program is defined as a sequence of function definitions. A function definition consists of a function name, a signature, and a body block. A signature includes a lifetime preorder, a list of arguments, and a return type. Here, a lifetime preorder is a set of lifetime variables and a preorder on them that presents the constraints on the lifetime variables. The lifetime preorder will also be used as a part of the type derivation context later. A block is a pair of a statement and a return value.

Types. The types of Qurts-core are defined in Figure 4. As we have seen in the examples, Qurts-core has linearity $\#^\alpha T$ and immutable reference $\&^\alpha T$. We also have product types $T_1 \times T_2$ and unit type $()$. For the lifetime, in addition to the lifetime variables $\alpha, \beta, \dots$, we have the shortest lifetime $\perp$, called ‘empty’, and the longest lifetime $\top$, called ‘static’. In ascii, α is written as ‘ $\mathbf{a}$ ’, $\perp$ as ‘ $\emptyset$ ’, and $\top$ as ‘ $\mathbf{static}$ ’. The empty lifetime is a new special lifetime which was not in Rust, and it is used to represent the linear type by $\#^\perp$.

Rust-like features. As we have seen in the examples, Qurts-core has explicit declarations of lifetimes. The statement $\text{newlft } \alpha$ starts a new lifetime variable α , and the statement $\text{endlft } \alpha$ ends it. Every statement $\text{newlft } \alpha$ must have a unique corresponding statement $\text{endlft } \alpha$ in the same scope. In full Qurts such statements should be automatically inferred if the programmer does not specify

them, but in the core language they are explicit. We also impose that every lifetime variable only lives once. That is, a lifetime variable cannot be restarted after it has ended, or more precisely, the following code is prohibited:

```
1 endlft 'a;
2 newlft 'a;
```

The statement $\alpha \leq \beta$ is used to add constraints on lifetimes (α ends before β). x as T is used to coerce the type of x to T . For example, we can coerce the linearity type $\#^\beta$ to $\#^\alpha$ when $\alpha \leq \beta$. We also have a statement $\text{let } y = \&^\alpha x$ to immutably borrow the variable x with lifetime α , drop x to drop the variable x , copy x to copy the variable x . These α , drop, and copy are also assumed to be inferred in full Qurts. As in Rust, to call a function, we need to assert that the lifetime of the arguments is valid. To do this, the correspondence between the lifetime variables in the function signature and the lifetime variables in the caller are assumed to be given explicitly in the core language. For simplicity, we do not allow a loop or a recursive function in the core language, so we assume all the well-typed functions terminates.

Quantum features. Qurts also has statements and expressions that are specific to quantum computing. The programmer can apply a *unitary gate* U to a list of qubits x with the syntax $U(x)$. We also have primitive isometric gates where the isometry is the *lift* of a classical Boolean injective function c on bits, given by special consideration with the syntax $[c](x)$. This includes the introduction of a new qubit $[0]$, X -gate, CX -gate, and so on. The syntax $\text{meas}(x)$ performs a *quantum measurement* on the qubit x in the computational basis; the value of this expression is the outcome of the measurement as a boolean value. Finally, Qurts has an analogue of the classical conditional statement with syntax $\text{qif } x \ B_1 \text{ else } B_0$. This expression evaluates to the execution of the blocks in both branches in superposition according to qubit x .

3.2.2 Type System.

Typing judgements. A judgement for a statement is defined by a relation on pairs of a type context and a lifetime preorder as $S: (\Gamma, \mathbf{A}) \rightarrow (\Gamma', \mathbf{A}')$, where $(\Gamma, \mathbf{A})$ is the context before the statement and $(\Gamma', \mathbf{A}')$ is the context after the statement. The typing context Γ is basically a list of variables and their types, but also annotates variables with *aliveness*, which can be active or $\dagger^\alpha$. By default a variable is active, written simply as $x: T$, and it becomes *frozen* ($\dagger^\alpha$) during the lifetime α when the variable is immutably borrowed. The lifetime preorder $\mathbf{A}$ represents the set of lifetimes that has not ended yet. Selected (simplified) typing rules for statements are shown in Figure 5.

Lifetimes. Introduction and elimination of lifetimes is governed by the rules **TYP NEW LFT** and **TYP END LFT**. The rule **TYP NEW LFT** simply adds a new lifetime α to the lifetime preorder $\mathbf{A}$. The rule **TYP END LFT** states that lifetime α can be ended if there is no other lifetime that has to end before α , and there is no reference $\&^\alpha$ in the typing context. After ending lifetime α , all frozen variables with lifetime α are *defrosted* by recovering the original context $x: T$ from $x: \dagger^\alpha T$.

The presence of lifetimes may make the Qurts-core typing rules look a bit complicated. For example, the premise of the rule **TYP FN CALL** for function calls verifies the lifetime constraints of the arguments by substituting all the lifetimes in the function signature with those in the caller. Once lifetimes are ignored, however, these rules are no different from the conventional ones.

The rule **TYP BORROW** governs borrowing a variable. This rule freezes the variable x with lifetime α and introduces a new variable y with the type $\&^\alpha T$. The premise ensures that no reference can be borrowed after the lifetime of the reference itself.

The rule **TYP DROP** governs dropping a variable from the context, and is critical for affine types with lifetime restrictions. The judgement $\mathbf{A} \vdash T: \text{Drop}$ in Figure 6 means that values of type T can

Typing Statements:

		$S: (\Gamma, \mathbf{A}) \rightarrow (\Gamma', \mathbf{A}')$	
TYP NEW LFT		TYP END LFT	
$\text{newlft } \alpha: (\Gamma, \mathbf{A}) \rightarrow (\Gamma, \mathbf{A} \cup \{\alpha\})$		$\alpha \text{ is minimal in } \mathbf{A} - \{\perp\} \quad \&^\alpha \text{ does not appear in } \Gamma$	
		$\text{endlft } \alpha: (\Gamma, \mathbf{A}) \rightarrow (\text{defrost}_\alpha(\Gamma), \mathbf{A} - \alpha)$	
		$\text{defrost}_\alpha(x: {}^aT) := \begin{cases} x: T & (\mathbf{a} = \dagger\alpha) \\ x: {}^aT & (\text{otherwise}) \end{cases}$	
TYP FN CALL			
		$\Sigma_g = \langle \beta_0, \dots, \beta_{m-1} \mid \beta_{a_0} \leq \beta_{b_0}, \dots, \beta_{a_{l-1}} \leq \beta_{b_{l-1}} \rangle (x'_0: S_0, \dots, x'_{n-1}: S_{n-1}) \rightarrow S_n$	
		$\forall j \in \{0, \dots, l-1\}, \alpha_{a_j} \leq \alpha_{b_j} \in \mathbf{A} \quad \forall i \in \{0, \dots, n\}, T_i = S_i[\alpha_0/\beta_0, \dots, \alpha_{m-1}/\beta_{m-1}]$	
		$\text{let } y = g(\alpha_0, \dots, \alpha_{m-1})(x_0, \dots, x_{n-1}): (\Gamma + \{x_0: T_0, \dots, x_{n-1}: T_{n-1}\}, \mathbf{A}) \rightarrow (\Gamma + \{y: T_n\}, \mathbf{A})$	
TYP BORROW		TYP DROP	
$\forall \gamma \in \{\alpha \mid \&^\alpha \text{ appear in } T\}, \alpha \leq \gamma \in \mathbf{A}$		$\mathbf{A} \vdash T: \text{Drop}$	
$\text{let } y = \&^\alpha x: (\Gamma + \{x: T\}, \mathbf{A}) \rightarrow (\Gamma + \{y: \&^\alpha T, x: \dagger^\alpha T\}, \mathbf{A})$		$\text{drop } x: (\Gamma + \{x: T\}, \mathbf{A}) \rightarrow (\Gamma, \mathbf{A})$	
TYP MEAS			
		$\text{let } y = \text{meas}(x): (\Gamma + \{x: \#^a \text{qbit}\}, \mathbf{A}) \rightarrow (\Gamma + \{y: \#^T \text{bool}\}, \mathbf{A})$	
TYP UNITARY			
		$U \in \text{Unitary}(n)$	
		$\text{let } y = U(x): (\Gamma + \{x: \#^a \text{qbit}^n\}, \mathbf{A}) \rightarrow (\Gamma + \{y: \#^{\perp} \text{qbit}^n\}, \mathbf{A})$	
TYP LIFTED			
		$c: \{0, 1\}^n \rightarrow \{0, 1\}^m; \text{injection}$	
		$\text{let } y = [c](x): (\Gamma + \{x: \#^a \text{qbit}^n\}, \mathbf{A}) \rightarrow (\Gamma + \{y: \#^a \text{qbit}^m\}, \mathbf{A})$	
TYP QIF			
$x: \&^\alpha \text{qbit} \in \Delta \quad \alpha \in \mathbf{A} \quad S_i: (\Gamma, \mathbf{A}) \rightarrow (y_i: T, \mathbf{A}) \quad S_i: \text{PQ} \quad T: \text{PQ}$			
		$\text{let } y = \text{qif } x \{ S_1; y_1 \} \text{ else } \{ S_0; y_0 \}: (\Gamma + \Delta, \mathbf{A}) \rightarrow ((y: \&^\alpha T) + \Delta, \mathbf{A})$	

Fig. 5. Selected typing rule for statements

Drop:

				$\mathbf{A} \vdash T: \text{Drop}$	
DROP BOOL	DROP BORROW	DROP OWN	DROP UNIT	DROP TUPLE	
$\mathbf{A} \vdash \text{bool}: \text{Drop}$	$\mathbf{A} \vdash \&^a T: \text{Drop}$	$\mathbf{a} \in \mathbf{A}$	$\mathbf{A} \vdash (): \text{Drop}$	$T_0: \text{Drop} \quad T_1: \text{Drop}$	
		$\mathbf{A} \vdash \#^a T: \text{Drop}$		$T_0 \times T_1: \text{Drop}$	

Fig. 6. Drop trait

be dropped at time $\mathbf{A}$. The rule **DROP BORROW** states that types $\&^\alpha T$ can be dropped at any time. On the other hand, **DROP OWN** states that the type $\#^a T$ can only be dropped until the lifetime $\mathbf{a}$ ends.

Quantum operations. The next three typing rules **TYP MEAS**, **TYP UNITARY** and **TYP LIFTED** govern basic quantum operations: measurements, unitary gates and lifted isometries. All three expressions take qubit(s) bound with $\#^a$ by lifetime $\mathbf{a}$ as an argument, but their return types are different. The return type of a measurement is $\#^T \text{bool}$: a classical boolean that can always be dropped. General unitary gates have a linear return type $\#^{\perp} \text{qbit}^n$, because the output is not uncomputable. Lifted isometries have return type $\#^a \text{qbit}^m$ and remain affine if the input is.

The most complicated and critical typing rule for Qurts-core is the rule **TYP QIF** governing qif statements. This rule only applies when no measurement occurs under quantum control, and no

Purely Quantum: $_ : \text{PQ}$

PQ TY BASE qbit: PQ	PQ TY OWN $T : \text{PQ}$	PQ TY TUPLE $T_0 : \text{PQ} \quad T_1 : \text{PQ}$	PQ EXPR $e \neq \text{meas } x \wedge ((e = f(\bar{\alpha})(\bar{x}) \wedge \text{fn-name}(F) = f) \Rightarrow F : \text{PQ})$
	$\#^a T : \text{PQ}$	$T_0 \times T_1 : \text{PQ}$	$e : \text{PQ}$
	PQ STMT S does not include non-PQ expression	PQ BLOCK $S : \text{PQ}$	PQ FN $B : \text{PQ}$
	$S : \text{PQ}$	$\{S; x\} : \text{PQ}$	$\text{fn } f \text{ A } \Gamma \rightarrow T \text{ B} : \text{PQ}$

Fig. 7. Purely Quantum: types without classical data and programs without measurements

Typing Block, Function:

TYP BLOCK $S : (\Gamma, \mathbf{A}) \rightarrow (x : T, \mathbf{A})$ $(\Gamma, \mathbf{A}) \vdash \{S; x\} : T$	TYP FN $F = \text{fn } f \Sigma B \quad \Sigma = \langle \alpha_0, \dots, \alpha_{m-1} \mid \alpha_{a_0} \leq \alpha_{b_0}, \dots, \alpha_{a_{l-1}} \leq \alpha_{b_{l-1}} \rangle \Gamma \rightarrow T$ $\mathbf{A} : \text{the smallest preorder on } \{\alpha_i\} \text{ including } \{\alpha_{a_j} \leq \alpha_{b_j}\} \quad (\Gamma, \mathbf{A}) \vdash B : T$ $\vdash F : \text{Function}$
---	--

Fig. 8. Typing rule for blocks and function

‘superposition of classical data’ is created in the return type. These properties are checked by the judgement $S_i : \text{PQ}$ and $T : \text{PQ}$ (Purely Quantum) in Figure 7. The former means that the statement does not include any measurement, which is close to Silq’s `mfree` [Bichsel et al. 2020]. The latter means that the type does not include any booleans or references. It is also important to note that the type of the return value y is not the type T of the two branches, but must be $\#^a T$ with the lifetime α of the reference x .

Block, Function. The typing rule for blocks and functions are shown in Figure 8. The rule **TYP BLOCK** for typing a block $\{S; x\}$ states that there must be only one variable x in the context after evaluating S for linearity. The rule **TYP FN** for typing a function $\text{fn } f \Sigma B$ states that the function f is well-typed if the body B is well-typed under the context $(\Gamma, \mathbf{A})$ generated from the signature Σ .

4 Simulation Semantics

This section discusses the first of the two small-step operational semantics, which we call *simulation semantics*. It represents a computation as a transition relation on the environment, including the state of quantum and classical memories. From now on, we assume all the programs we present are well-typed and equipped with their type derivation tree including the lifetimes.

The most notable rule of the simulation semantics is the one governing drop. When a variable is dropped, the memory locations owned by the variable are *deallocated*. The deallocation of the last qubit of the quantum memory is defined as the operation that converts the state $|\varphi_0\rangle|0\rangle + |\varphi_1\rangle|1\rangle$ to $|\varphi_0\rangle + |\varphi_1\rangle$. For general states $|\varphi_i\rangle$, this operation is neither unitary nor norm-non-increasing, and therefore not physically realisable. For example, if $|\varphi_0\rangle = |\varphi_1\rangle = \frac{1}{\sqrt{2}}|0\rangle$, then the norm of the initial state is $|\frac{1}{\sqrt{2}}|00\rangle + \frac{1}{\sqrt{2}}|01\rangle| = 1$, while the norm of the final state is $|\frac{1}{\sqrt{2}}|0\rangle + \frac{1}{\sqrt{2}}|0\rangle| = |\sqrt{2}|0\rangle| = \sqrt{2} > 1$. For orthogonal states $\{|\varphi_i\rangle\}$ physicality is restored, as the dropping operation preserves norms.

Section 4.3 below proves that the type system guarantees that the evaluation step of a well-typed program in a well-formed environment can only perform drop for a state satisfying the orthogonality condition. Therefore, the simulation semantics is norm-preserving for all cases except measurement.

$$\begin{array}{c}
\text{SIMULATION DROP} \\
\text{drop } x : (\Gamma, -) \rightarrow (-, -) \quad L_q = [l_q \in \text{Loc}_q \mid l_q \text{ owned by } x] \quad L_c = [l_c \in \text{Loc}_c \mid l_c \text{ owned by } x] \\
|\varphi\rangle = \sum_{i \in \{0,1\}^{|L_q|}} |\varphi_i\rangle |i\rangle \in \mathcal{H}_{\text{others}} \otimes \mathcal{H}_{L_q} \quad s = (s_0, s_1) \in \{0, 1\}^{\text{others}} \times \{0, 1\}^{L_c} \\
\hline
(\text{loc}, |\varphi\rangle, s) \xrightarrow{\text{drop } x} (\text{loc}[x \mapsto []], \sum_{i \in \{0,1\}^{|L_q|}} |\varphi_i\rangle, s_0) \\
\\
\text{SIMULATION COPY} \\
L_c = \{l_c \in \text{Loc}_c \mid l_c : \text{owned by } x\} \quad L'_c : \text{fresh classical locations} \quad L' = \text{loc}(x)[L'_c/L_c] \quad t = s(L_c) \\
\hline
(\text{loc}, |\varphi\rangle, s) \xrightarrow{\text{let } y = \text{copy } x} (\text{loc}[y \mapsto L'], |\varphi\rangle, s[t \leftarrow L'_c]) \\
\\
\text{SIMULATION QIF} \\
S = (\text{let } y = \text{qif } x \{ S_1; x_1 \} \text{ else } \{ S_0; x_0 \}) \quad \text{loc}(x) = [l_x] \quad L_i = \text{loc}_i(x_i) \quad |L| = |L_i| \quad L : \text{fresh} \\
(\text{loc}, |0\rangle |\varphi_0\rangle, s) \xrightarrow{S_0} (\text{loc}_0, |0\rangle |\psi_0\rangle \in \mathcal{H}_{l_x} \otimes (\mathcal{H}_{L_0} \otimes \mathcal{H}_{\text{others}}), s') \\
(\text{loc}, |1\rangle |\varphi_1\rangle, s) \xrightarrow{S_1} (\text{loc}_1, |1\rangle |\psi_1\rangle \in \mathcal{H}_{l_x} \otimes (\mathcal{H}_{L_1} \otimes \mathcal{H}_{\text{others}}), s') \\
\hline
(\text{loc}, \sum_{i \in \{0,1\}} |i\rangle |\varphi_i\rangle, s) \xrightarrow{S} (\text{loc}_0[x_0 \mapsto [], y \mapsto L], \sum_{i \in \{0,1\}} |i\rangle |\psi_i\rangle, s')
\end{array}$$

Fig. 9. Selected simulation semantics rules.

4.1 Simulation Semantics Definition

All variables are considered to be pointers: states of quantum and classical memory are stored in locations, and variables are mapped to memory locations.

Let Loc_q and Loc_c be the sets of locations for quantum and classical memories. An *environment* is a triple $(\text{loc}, |\varphi\rangle, s)$ of a map $\text{loc} : \text{Var} \rightarrow \text{List}(\text{Loc}_c + \text{Loc}_q)$ from variables to locations, $|\varphi\rangle \in \mathcal{H}_{L_q} ::= \bigotimes_{l \in L_q} \mathbb{C}^2$ and $s \in \{0, 1\}^{L_c}$ are the states of quantum and classical memories, and $L_q \subset \text{Loc}_q$ and $L_c \subset \text{Loc}_c$ are the image of loc .

An environment $(\text{loc}, |\varphi\rangle, s)$ is *compatible* with a context Γ when $\text{loc}(x)$ has the form $\llbracket T \rrbracket$ for all the variables $x : {}^a T$ in Γ , where $\llbracket T \rrbracket$ is defined as follows. (‘+’ denotes the concatenation of lists.)

$$\begin{array}{ll}
\llbracket \text{qbit} \rrbracket = [l_c], & \llbracket \text{bool} \rrbracket = [l_c], \\
\llbracket \#^a T \rrbracket = \llbracket T \rrbracket, & \llbracket \&^\alpha T \rrbracket = \llbracket T \rrbracket, \\
\llbracket () \rrbracket = [], & \llbracket T_1 \times T_2 \rrbracket = \llbracket T_1 \rrbracket + \llbracket T_2 \rrbracket.
\end{array}$$

Each statement $(\Gamma, A) \xrightarrow{S} (\Gamma', A')$ gives a transition relation $e \xrightarrow{S} e'$ on compatible environments.

To define the semantics of the statements *drop* and *copy*, we first discuss *ownership* of the locations. Let e be an environment compatible with a type context Γ , and $x : T$ be a variable in that context. When a location l is included in $\text{loc}(x)$, and the corresponding appearance of basic type *qbit* or *bool* in T is not bound by any $\&^\alpha$, then we say that l is *owned* by x .

Selected rules are presented in Figure 9, all (other) rules are defined in [Hirata and Heunen 2024, Appendix C].

Drop. The transition step **SIMULATION DROP** for **drop** x forgets the variable x from the environment and drops the value stored in the memory locations owned by x . Note that when the variable x includes a reference of a location l but does not own it, the location l is not deallocated.

Copy. The rule **SIMULATION COPY** defines the transition step for **copy**(x). When the variable x is copied to y , it copies the classical memory owned by x to y , as well as all references x has.

line		loc: $\text{Var} \rightarrow \text{List}(\text{Loc}_q + \text{Loc}_c)$	quantum state	classical state
		$[],$	$1,$	$()$
1	$\xrightarrow{\text{let } x_0 = [\emptyset]();}$	$[x_0 \mapsto [l_x]],$	$ 0\rangle,$	$()$
2,3	$\xrightarrow{\text{let } x_1 = H(x_0);}$	$[x_1 \mapsto [l_x]],$	$ +\rangle = \frac{1}{\sqrt{2}} 0\rangle + \frac{1}{\sqrt{2}} 1\rangle,$	$()$
4,5	$\xrightarrow{\text{let } r = \&x_1;}$	$[x_1 \mapsto [l_x], r \mapsto [l_x]],$	$ +\rangle,$	$()$
6,7	$\xrightarrow{\text{let } y = \text{qif } r \{ \dots \}}$	$[x_1 \mapsto [l_x], r \mapsto [l_x], y \mapsto [l_y]],$	$\frac{1}{\sqrt{2}} 00\rangle + \frac{1}{\sqrt{2}} 11\rangle,$	$()$
8	$\xrightarrow{\text{drop } r;}$	$[x_1 \mapsto [l_x], y \mapsto [l_y]],$	$\frac{1}{\sqrt{2}} 00\rangle + \frac{1}{\sqrt{2}} 11\rangle,$	$()$
9,10	$\xrightarrow{\text{drop } y;}$	$[x_1 \mapsto [l_x]],$	$ +\rangle,$	$()$
11	$\xrightarrow{\text{let } \text{ret} = \text{meas}(x_1);}$	$[b \mapsto [l_b]],$	$\frac{1}{\sqrt{2}},$	(0)
11	$\xrightarrow{\text{let } \text{ret} = \text{meas}(x_1);}$	$[b \mapsto [l_b]],$	$\frac{1}{\sqrt{2}},$	(1)

Fig. 10. Toy example: evaluation steps of the simulation semantics.

Quantum If. The most subtle rule in the simulation semantics is perhaps that governing **qif**. The semantics of the whole quantum if statement is the sum of the resulting state of the two blocks $\{S_i; x_i\}$ of the quantum if statement in **SIMULATION QIF**. The **Purely Quantum** rule guarantees that the state of the classical memory s remains unchanged during the evaluation of the blocks. As for the locations, since the two variables x_i might not have the same locations, we introduce a fresh set of locations L and replace the locations of x_i with L . Thus, the program **qif** $p \{ \text{let } t_1 = (y, x); t_1 \} \text{ else } \{ \text{let } t_0 = (x, y); t_0 \}$ is interpreted as a controlled-swap operation.

For the post-environment, the next lemma states that the locations loc_0 and loc_1 only differ in the locations owned by x_i , proving that this rule is not biased towards either branch.

LEMMA 4.1. *The map $\text{loc}_i[x_i \mapsto [], y \mapsto L]$ in the **SIMULATION QIF** does not depend on i .*

4.2 Example

Consider again the **toy example** from Section 3.1. We will use it to illustrate how the simulation semantics works. Say that the environment starts at $e = (\text{loc}, v, s)$ with initial state $(\text{loc} = \lambda_.[], 1, ())$. The program then evaluates as in Figure 10. In line 1, the quantum memory x_0 is initialized to $|0\rangle$. In line 2, the type of x_0 is coerced from $\#^T \text{qbit}$ to $\#^1 \text{qbit}$, making x_0 linearly owned to be able to apply the Hadamard gate to it in line 3. Lines 4 and 5 declare a new lifetime α , and borrow the variable x_1 to make a reference r of type $\#^\alpha \text{qbit}$. Lines 6 and 7 are evaluated as the superposition of the following transition steps for $i = 0, 1$.

$$\xrightarrow{\text{let } z = [i]();} \begin{array}{ll} [r \mapsto [l_x]], & \frac{1}{\sqrt{2}}|i\rangle, \quad () \\ [r \mapsto [l_x], z \mapsto [l_z]], & \frac{1}{\sqrt{2}}|ii\rangle, \quad () \end{array}$$

In line 8, we drop the reference r , which leaves the memory unaltered. On the other hand, dropping the variable y deallocates the memory location l_y that y owned. Line 11 measures the quantum memory x_1 , resulting in two possible branches of transitions. The two resulting environments have different classical states, each with probability $\frac{1}{2}$.

4.3 Properties

Next, we investigate properties of the simulation semantics. For the proof, see [Hirata and Heunen 2024, Appendix C]. The first one is progress.

LEMMA 4.2. *Let S be a statement, e be an environment, and $S: (\Gamma, \mathbf{A}) \rightarrow (\Gamma', \mathbf{A}')$ be a well-typed statement. If e is compatible with Γ , then there exists an compatible environment e' such that $e \xrightarrow{S} e'$.*

However, as we have seen in the example at the beginning of Section 4, compatibility of the environment does not guarantee physical realisability of the state. We need to take into account types, especially whether they are affine or linear, and consider only certain *well-formed* environments.

Definition 4.3. Let $\mathbf{A}$ be a lifetime preorder and $x: {}^aT$ in type context Γ . A location is:

- *affinely owned* if $x: {}^aT$ owns it and the corresponding basic type in T is only bounded by $\#^a$ such that $0 < a \in \mathbf{A}$; otherwise it is *linearly owned*;
- *frozen* when it is owned by a frozen variable $x: {}^{\dagger a}T$ or not owned by any variable at all.

Definition 4.4. A triple of a type context Γ , a lifetime preorder $\mathbf{A}$, and an environment e , is *well-formed* if the following conditions are satisfied:

- the environment e is compatible with Γ ;
- any location $l \in L_q + L_c$ is owned by at most one variable in Γ ;
- the state of the quantum memory in $\mathcal{H}_f \otimes \mathcal{H}_l \otimes \mathcal{H}_a$ is of the form

$$\sum_{i \in \{0,1\}^n} |i\rangle |\varphi_i\rangle |f(i)\rangle$$

for a function $f: \{0,1\}^n \rightarrow \{0,1\}^m$, where $\mathcal{H}_f$, $\mathcal{H}_l$, and $\mathcal{H}_a$ are the Hilbert spaces of memory locations which are frozen, linearly owned, and affinely owned.

Let us explain the last condition for well-formedness. The space $\mathcal{H}_f$ contains quantum memory locations that are immutably referenced or outside the scope, and the space $\mathcal{H}_a$ is the space of locations that are owned by some variable in the context, which could be dropped in the future. The space $\mathcal{H}_l$ contains the remaining locations, with no information on their state. The existence of a function $f: \{0,1\}^n \rightarrow \{0,1\}^m$ says that frozen locations determine which part of the quantum memory can be deallocated. This condition also guarantees that deallocation is norm-preserving:

$$\left| \sum_{i \in \{0,1\}^n} |i\rangle |\varphi_i\rangle |f(i)\rangle \right|^2 = \sum_{i \in \{0,1\}^n} \left| |\varphi_i\rangle \right|^2 = \left| \sum_{i \in \{0,1\}^n} |i\rangle |\varphi_i\rangle \right|^2.$$

For each environment e , we define the *probability* of the environment e by $\|e\|^2 := \|\varphi\|^2$ where $|\varphi\rangle$ is the quantum state of e .

THEOREM 4.5. *Let $\text{fn } f \mathbf{A} \Gamma \rightarrow T \{ S; x \}$ be a well-typed function, and E be the set of environments $\{e' \mid e \xrightarrow{S} e'\}$. If $(\Gamma, \mathbf{A}, e)$ is well-formed, then so is $(x: T, \mathbf{A}, e')$ for each $e' \in E$, and the sum of the probabilities of environments in E is equal to the probability of e :*

$$\|e\|^2 = \sum_{e' \in E} \|e'\|^2.$$

5 Uncomputation Semantics

This section concerns a second operational semantics for Qurts, called uncomputation semantics, which uses reversible pebble games. The uncomputation semantics compiles a Qurts program into a circuit graph, and the execution of the program is defined by the play of the pebble game.

The simulation semantics performed uncomputation immediately after a drop statement, but this strategy is not efficient in terms of time or space as a quantum circuit. The uncomputation semantics, on the other hand, does not follow any fixed evaluation strategy. Instead, a program is interpreted by a circuit graph and a pebble game play on it, where the latter is non-deterministic.

Thus, different pebbling strategies give rise to different execution sequences, with better strategies yielding more efficient quantum circuits.

Even though different strategies may induce different circuits, Theorem 5.3 below proves that the resulting quantum state is the same for any strategy. We also show that simulation semantics and uncomputation semantics are equivalent in Theorem 5.4.

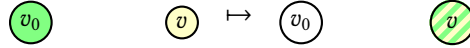
5.1 Reversible Pebble Game for Qurts

The pebble game we use for Qurts has some differences from the original reversible pebble game [Bennett 1989]. In the original, the rule of putting a pebble on w from the pebbling state $(v \rightarrow w)$ corresponds to an application of a Controlled- U_f gate to an auxiliary qubit initialized to $|0\rangle$ as follows, where $U_f: \mathbb{C}^{2^n} \rightarrow \mathbb{C}^{2^n}$ is the unitary lifting a bijection $f: \{0, 1\}^n \rightarrow \{0, 1\}^n$.

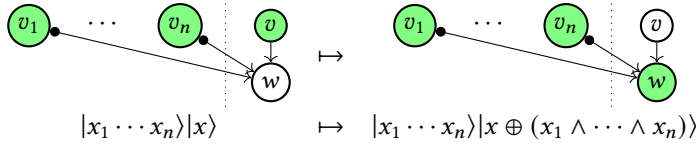
$$|x\rangle \mapsto |x\rangle|0\rangle \mapsto |x\rangle|f(x)\rangle \in \mathcal{H}_v \otimes \mathcal{H}_{\text{auxiliary}}$$

On the other hand, to stay faithful to Rust's move semantics, the default execution should be interpreted as an application of U_f to the same qubit, as $|x\rangle \mapsto |f(x)\rangle \in \mathcal{H}_v$. This has a corresponding pebbling state $(v \rightarrow w)$. Also, we should still have a rule to introduce an auxiliary qubit. Therefore, we replace the original rule of putting a pebble on w by three new rules: *init*, *copy* and *gate*.

The *init* rule is for the special *init vertices* in the circuit graph, corresponding to the state $|0\rangle$. The *init* rule allows to add or remove a pebble on any init vertex. The *copy* rule allows moving a pebble from the initial vertex to a vertex v which is already pebbled. Note that multiple pebbles on the same vertex are allowed. This move corresponds to the application of the controlled-not gate, converting the state $|0i\rangle \in \mathcal{H}_{\text{green}} \otimes \mathcal{H}_{\text{yellow}}$ to $|ii\rangle$.



The rule *gate* is for single target gates, which allows to move a pebble from v to w as in the following diagram. The edge from v to w represents the target of the gate, and the other edges from v are the controls. This rule only applies when all controls already have a pebble.



Together, these three rules can simulate the original rule of putting a pebble on w by applying the *init*, *copy*, and *gate* rules in sequence.

A further new *split* rule governs the quantum if statement. Consider the following Qurts code:

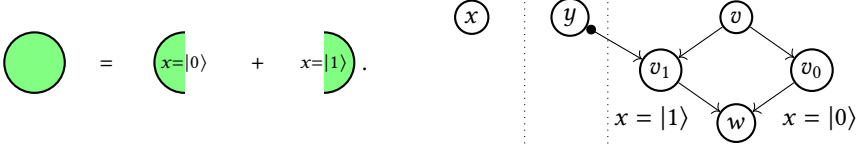
```
qif x { [cnot](y, t) } else { (y, [not](t)) }
```

In a quantum if statement, it should not matter which branch is executed first. Equivalently, the applications of two gates $CCX(x, y, t)$ and $C^\perp X(x, t)$ commute, where the latter is the negatively-controlled-X gate. To express this commutativity as a circuit graph and a pebbling strategy, we allow *splitting a pebble in half* and introduce a *merge vertex with a guard*.

First, for each vertex x , we add a rule to split a pebble into half pebbles conditioned on $x = |0\rangle$ and $x = |1\rangle$ as in the left diagram below. The idea is that a pebble conditioned on $x = |1\rangle$ is used in the then branch, and the other is used in the else branch of *qif* x . In order to move these fragments, the vertex x must be pebbled since the semantics of the movement uses x as a control. These two fragments can be moved independently, which enables executing the branches in parallel.

The graph on the right below represents the code above. The vertex v corresponds to the state of t , and vertices x, y correspond to the state of the variables x, y . The vertices v_0 and v_1 correspond to

the application of the gates in the branches of the quantum if statement, for which w is the *merge* vertex. There are two incoming edges, from v_0 and v_1 , through which only pebbles fragmented by the guards $x = |0\rangle$ respectively $x = |1\rangle$ can pass.



This splitting rule also allows to handle a situation when a drop appears inside a quantum if statement. By this splitting rule, for example, one of the possible pebbling strategies for the following code says that this is equivalent to a Toffoli gate. See [Hirata and Heunen 2024, Appendix D] for the detailed strategy.

```
let (x, y) = [cnot](x, |0>);
let y' = qif z { y } else { drop y; |0> }
```

$$|ij0\rangle \mapsto |iji\rangle \mapsto \begin{cases} |i1i\rangle & \text{if } j = 1, \\ |i00\rangle & \text{if } j = 0. \end{cases}$$

5.2 Uncomputation Semantics

We are ready to describe the Qurts uncomputation semantics, leaving the details to [Hirata and Heunen 2024, Appendix D].

5.2.1 Circuit Graphs. Without loss of generality, we assume that all the lifted isometries $[c]$ are decomposed into $|0\rangle$ or single-target gates, denoted as $\text{let } y = |0\rangle$, $\text{let } y' = \neg y$, or $\text{let } y' = y \oplus (\neg^{i_1} x_1 \wedge \dots \wedge \neg^{i_n} x_n)$. Note that negation applies the uncontrolled single-target gate X . The following definition of circuit graphs and pebbles extends Unqomp [Paradis et al. 2021].

Definition 5.1 (Circuit Graph). A *circuit graph* $G = (V, E)$ is a directed graph with a partition $V = V_{\text{init}} \sqcup V_{\text{linear}} \sqcup V_{\text{gate}} \sqcup V_{\text{merge}}$ of the vertices and a function $\text{loc}: V \rightarrow \text{Loc}_q$, such that:

- only a vertex in $v \in V_{\text{gate}}$ can have incoming edges with different locations;
- a vertex v is in V_{init} if and only if it does not have any incoming edge, and it is unique in the subgraph G_l induced by $\{v \mid \text{loc}(v) = l\}$ for each location l ;
- a vertex $v \in V_{\text{gate}}$ has a unique incoming edge $w \rightarrow v$ such that $\text{loc}(v) = \text{loc}(w)$;
- a vertex $v \in V_{\text{merge}}$ has exactly two ordered incoming edges e_0 and e_1 ; the vertex v has an associated pair $g = (w, \alpha)$ of a vertex w and a lifetime α , called the *guard* of v ;
- a vertex v in V_{linear} is unique in G_l for each l ; and each edge with target in V_{linear} has a formal conjunction of guards $g_1 \wedge \dots \wedge g_n$.

A vertex in V_{init} has state $|0\rangle$, and a vertex in V_{gate} corresponds to applying a single-target gate. The vertex that corresponds to the X -gate applied to v is denoted by $\neg v$. The new parts of the definition are V_{merge} and V_{linear} .

Vertices V_{merge} model the quantum if statement according to the previous subsection. The lifetime α of a guard represents the restriction that ‘passage is allowed only during this period’, reflecting the intuition that variables can only be (un)computable during their lifetime.

Vertices in V_{linear} model locations becoming linear when pebbled instead of affine. Intuitively, this restricts attention to the forward closed subgraph of gates applied to linearly owned locations. Vertices in V_{linear} also have guards, because they may later need to play a role similar to V_{merge} vertices when the location becomes linearly owned under quantum control.

Definition 5.2. Let G be a circuit graph. A *guard* $g = (v, \alpha)$ is a vertex v with a lifetime α , and a *pebble* $p\{g_1 \wedge \dots \wedge g_n\}$ is a formal conjunction of guards with a label $p \in \text{Label}$. The pebble $p\{\top\}$ is called *whole* if $n = 0$ and *fragmented* otherwise. Pebbles may be added by $p\{g \wedge g'_1 \wedge \dots \wedge g'_n\} + p\{\neg g \wedge g'_1 \wedge \dots \wedge g'_n\} = p\{g'_1 \wedge \dots \wedge g'_n\}$, where the negation of a guard is $\neg(v, \alpha) = (\neg v, \alpha)$.

$$\begin{array}{c}
\text{SCHEDULER UNCOMPUTER} \\
\frac{\mathbf{A}, G \models (|\varphi\rangle, \text{peb}) \Rightarrow (|\varphi'\rangle, \text{peb}')}{(\bar{\Theta}, \mathbf{A}, G, (|\varphi\rangle, s), \text{peb}) \rightarrow (\bar{\Theta}, \mathbf{A}, G, (|\varphi'\rangle, s), \text{peb}')} \\
\\
\text{SCHEDULER EXEC} \\
\frac{G, \text{peb} \models \Theta: \text{executable} \quad (\Theta, \mathbf{A}, (|\varphi\rangle, s), G) \rightarrow (\bar{\Theta}', \mathbf{A}', (|\varphi'\rangle, s'), G')}{((\bar{\Theta}, \Theta), \mathbf{A}, G, (|\varphi\rangle, s), \text{peb}) \rightarrow ((\bar{\Theta} + \bar{\Theta}'), \mathbf{A}', G', (|\varphi'\rangle, s'), \text{peb})} \\
\\
\text{SCHEDULER AWAIT} \\
\frac{\text{state}(t) = \mathbf{Await}(x, t_1, \dots, t_m) \quad \text{state}(t_i) = \mathbf{Terminated}(\Gamma_i) \quad (\Theta_t, G) \xrightarrow{(x, \mathbf{A}, \Theta_{t_1}, \dots, \Theta_{t_m})} (\Theta'_t, G')}{((\bar{\Theta}, \Theta_t, \bar{\Theta}_{t_i}), \mathbf{A}, G, (|\varphi\rangle, s), \text{peb}) \rightarrow ((\bar{\Theta}, \Theta'_t), \mathbf{A}, G', (|\varphi'\rangle, s'), \text{peb})}
\end{array}$$

Fig. 11. Uncomputation semantics

A *pebbling of the circuit graph* $\{\text{peb}(v)\}_{v \in V}$ is an assignment of a set of pebbles to each $v \in V$.

The label $p \in \text{Label}$ identifies a pebble $p\{g_1 \wedge \dots \wedge g_n\}$ and represents a qubit. The vertex v in the guard indicates that the pebble is split by the condition $v = |1\rangle$. For instance, say vertex v has a pebble $q\{\top\}$. When we have one fragmented pebble $p\{(v, \mathbf{a})\}$ on a vertex in state $|1\rangle$ and another fragmented pebble $p\{(\neg v, \mathbf{a})\}$ on a vertex in state $|0\rangle$, then the whole quantum state is $\gamma_0|00\rangle + \gamma_1|11\rangle \in \mathcal{H}_{q,p}$ ($\gamma_i \in \mathbb{C}$). A pebble can have multiple guards representing qubit states in nested quantum if statements. Just like merge vertices, pebbles also have lifetimes: a pebble $p\{(v_1, \mathbf{a}_1) \wedge \dots \wedge (v_n, \mathbf{a}_n)\}$ can be moved only during the lifetimes $\mathbf{a}_1, \dots, \mathbf{a}_n$.

5.2.2 Definition of Uncomputation Semantics. Uncomputation semantics is a small-step operational semantics. A program execution is represented by a relation $\rightarrow$ on tuples $(\bar{\Theta}, \mathbf{A}, G, (|\varphi\rangle, s), \text{peb})$ of *threads* $\bar{\Theta}$, a lifetime preorder $\mathbf{A}$, a circuit graph G , a state of the memory $(|\varphi\rangle, s)$, and a pebbling peb . There are two main rules, **SCHEDULER UNCOMPUTER** and **SCHEDULER EXEC**, that we now describe. Refer to Figure 11, and to [Hirata and Heunen 2024, Appendix D] for formal definitions and proofs.

Uncomputer. The judgement $\mathbf{A}, G \models (|\varphi\rangle, \text{peb}) \Rightarrow (|\varphi'\rangle, \text{peb}')$ for the play of the pebble game is called *uncomputer execution*, and is used in the rule **SCHEDULER UNCOMPUTER**. This rule can change the state of the pebbling, but not the circuit graph.

Thread. Execution of qif statements ideally evaluate both branches in parallel. Therefore, we interpret a qif statement as spawning two *threads*, one for each branch. Each thread has its own statement to execute, and the execution of a thread is presented by a relation $(\Theta, \mathbf{A}, (|\varphi\rangle, s), G) \rightarrow (\bar{\Theta}', \mathbf{A}', (|\varphi'\rangle, s'), G')$ where Θ is a thread and $\bar{\Theta}'$ is a set of threads obtained by the execution. Note that thread execution does not refer to the pebbling state, and its main role is to extend the circuit graph. However, there are some cases in which uncomputer execution and thread execution need to be synchronised, namely $\text{endlft } \alpha$, $x \text{ as } T, U(x)$, and $\text{meas}(x)$. This synchronisation is accomplished by the judgement $G, \text{peb} \models \Theta: \text{executable}$, which checks whether the thread Θ is executable with pebbling state peb . Finally, the rule **SCHEDULER AWAIT** awakes the parent thread after termination of the child threads.

5.2.3 Properties. Uncomputation semantics is non-deterministic in two ways: the order of execution between threads, and between a thread and the uncomputer. The next theorem states that the resulting quantum state is independent of the order of execution.

THEOREM 5.3. *The uncomputation semantics is thread-safe in the sense that all the execution orders of threads and uncomputer give the same state of the classical and quantum memory.*

Simulation semantics tries to deallocate qubits eagerly, as soon as the drop statement is called. The next theorem shows that we can simulate the simulation semantics by a pebbling strategy clearing corresponding pebbles eagerly.

THEOREM 5.4. *There is a pebbling strategy that simulates the simulation semantics. Therefore, uncomputation semantics and simulation semantics are equivalent.*

The previous two theorems show that any pebbling strategy gives the desired result of the computation described in the simulation semantics.

6 Related Work

This section compares and contrasts this article with other works, starting with the most relevant.

Silq [Bichsel et al. 2020] was the initial work attempting to create a type system for automatic uncomputation in quantum programming languages. *Silq* has some special type annotations `const` and `qfree`. The annotation `const` is similar to immutable reference in *Qurts*, which can only be annotated to arguments. In their paper [Bichsel et al. 2020], they mentioned that the `const` is related to immutable reference in *Rust*, but they concluded that the type system of *Rust* is not expressive enough to capture the subtlety of automatic uncomputation. They mentioned that the annotation `qfree` to functions, which ensures that the function is made of classical boolean circuits, is a feature that is not present in *Rust*, and together with the annotation `const`, they provide a type system for automatic uncomputation.

The annotation `qfree` in *Silq* and the type `#'a qbit` in *Qurts* are similar in that they are for safe uncomputation, but they are different in the following aspects. First, *Silq* does not have a concept like lifetime, but this is not the only difference. In *Silq*, the annotation `qfree` is for entire functions, and it ensures that the function body is made of classical boolean circuits. Therefore, in the following *Silq* function, there is no way to indicate that the second output qubit can be safely uncomputed but the first cannot. In *Qurts*, the type `#'static qbit` ensures that the second qubit can be safely uncomputed at any time as in the right.

```
1 // B = type for qubit in Silq
2 def f(x:B) : B x B { // not qfree
3   a := H(x);
4   return (a, 0:B);
5 }
```

Silq

```
1 fn f(mut x: qbit) -> (qbit, #'static qbit)
2 {
3   x.H();
4   return (x, |0>)
5 }
```

Qurts

In addition, it is not possible to define the function `forget` or `reinitialize` we presented in Section 3 in *Silq*. This problem is already known in [Lewis and Gehr 2021]. This is also because there is no way to annotate the type of qubits for inputs in *Silq* to ensure that it can be safely uncomputed.

Silq has imperative features such as assignments, which make the language puzzling. In Figure 12, we presented four functions in *Silq*. The function `f2` is obtained by inlining the variable `t` in line 4 to line 5 of `f1`, so the two functions should be equivalent. However, the former is rejected by the type checker. Similarly, the function `g2` is obtained by inlining the variable `t` in line 4 to line 5 of `g1`, but only the function `g1` is accepted by the type checker. This feature is not discussed in their paper [Bichsel et al. 2020] since they only discussed the core language *Silq-Core* that leaves out the imperative features. As discussed, the automatic uncomputation feature has a strong imperative aspect, which motivated the introduction of lifetime in type systems in *Qurts*. We do not know the specific reason why the type checker of *Silq* behaves this way, but it may be that *Silq*'s type system is not expressive enough for the imperative features. *Qurts* does not have this issue by definition since all intermediate variables are assumed to be bound by `let` in *Qurts-core*. See also [Hirata and Heunen 2024, Appendix E].

<pre> 1 def f1(f: const B ! -> qfree B) { 2 p := H(0:B); 3 r := 0:B; 4 t := f(p); // ERROR: variable 't' is // not consumed 5 if t { 6 r := X(r) 7 } 8 return (p,r) 9 } </pre>	<pre> 1 def f2(f: const B ! -> qfree B) { 2 p := H(0:B); 3 r := 0:B; 4 5 if f(p) { 6 r := X(r) 7 } 8 return (p,r) 9 } </pre>
<pre> 1 def g1(f: B ! -> qfree B) { 2 p := 0:B; 3 r := 0:B; 4 t := f(p); 5 if t { 6 r := X(r) 7 } 8 return r 9 } </pre>	<pre> 1 def g2(f: B ! -> qfree B) { 2 p := 0:B; 3 r := 0:B; 4 5 if f(p) { // ERROR: non-'lifted' quan- // tum expression must be consumed 6 r := X(r) 7 } 8 return r 9 } </pre>

Fig. 12. Silq: assignments and uncomputation

Unqomp [Paradis et al. 2021] and *Reqomp* [Paradis et al. 2024] are algorithms for automatic quantum uncomputation. Unqomp is an algorithm that prevents recomputation by delaying uncomputations to optimise time complexity, and reqomp is an algorithm that optimises number of qubits by allowing recomputation. Both algorithms are based on *circuit graphs*. They first construct a circuit graph from a quantum circuit which is known to be qfree, a circuit that can be represented as a classical Boolean function, and then construct another circuit graph with the uncomputation. In Reqomp, it is mentioned that pebble game is not suitable for the uncomputation problem, because the graph constructed is not expressive enough to represent non-qfree gates or repeatedly used values. The pebble game for Qurts overcomes these limitations: Our pebble game is not based on the dependency graph as in the original pebble game, but rather on the circuit graph defined in the paper Unqomp, which is more expressive. We can also represent repeatedly used values by using the same vertices in the graph, or have cyclic dependencies in the graph discussed being problematic in Unqomp. However, our type system asserts that the uncomputation is possible in some way even in such cases.

Qrisp. The Unqomp algorithm is incorporated in Qrisp [Seidel et al. 2023], a framework of Python modules and language extensions that replaces gates and qubits with functions and variables governed by a linear type discipline, and that compiles to quantum circuits.

The lazy approach of Unqomp saves computation time at the expense of a larger qubit overhead, which is often not desirable. Qrisp therefore extends qfree-ness to the concept of *permeability*, which allows more flexible resource management. The Qrisp compiler can automatically verify the correctness of the uncomputation, but this is no longer based on a type system. Instead, the compiler checks via operational semantics whether the unitary matrices in question are permeable. This compile-time simulation can be very costly. The type discipline of Qurts allows the flexibility of different pebbling strategies while maintaining compilation performance.

Modular synthesis. In parallel to and independent of this paper, the very recent work [Venev et al. 2024] has a similar goal. First, it presents an algorithm for uncomputation that benchmarks favourably against Unqomp. An interesting question is to which pebbling strategy this algorithm corresponds. Its second contribution is an intermediate representation that tracks data used for the uncomputation. It uses an effect system, similar to Silq’s qfree annotations, that gives sufficient conditions for the uncomputation algorithm to be applicable. In contrast, Qurts tracks similar data

using a type system that is a first class citizen, about which formal properties are proven, that is directly available to the programmer, and that is clean in the sense that it only extends the Rust type system with `#'a`.

ReQWIRE [Rand et al. 2018] is a similar compiler, whose main feature is that it has been verified with the interactive theorem prover Coq. It allows discarding qubits whose quantum state is verified to be $|0\rangle$ or $|1\rangle$ on a linear type system, so its goal is slightly different from automatic uncomputation.

Twist [Yuan et al. 2022] is a quantum programming language whose goal is to guarantee purity (the opposite of entanglement) by type discipline. This has impact on uncomputability, but that is not *Twist*'s main aim. The system enforces the type discipline using a combination of static analysis and runtime verification.

Proto Quipper [Fu et al. 2020] is a programming language and compiler that addresses uncomputation with a monadic type system that syntactically hides the discarded qubits from the programmer, but does not provide any guarantee or verification of correctness of the program.

Similarly, *ScaffCC* [JavadiAbhari et al. 2014, 2015] and *SQUARE* [Ding et al. 2020] are both compiler frameworks for quantum computation, that perform some analysis to optimise for quantum uncomputation, but unlike Qurts they are not guaranteed at the type level.

7 Future Work

We have shown that affine types with lifetime restrictions can usefully model quantum computation with automatic uncomputation. One might similarly look for other kinds of computation modeled by lifetime-restricted affine types than quantum computation. For example, such types could represent a situation where the garbage has to be taken out by the time the garbage collector arrives. Alternatively, swapping the roles of affine and linear types could model situations where garbage cannot be disposed of until the garbage bins are in place, or where people must remain imprisoned at a location until a certain time has elapsed.

Implementing a Qurts compiler is left to future work. From a syntactic perspective, the most interesting implementation challenges not yet present in Qurts-core include recursion, mutable variables and algebraic data types. Their interaction with the type $\#^{\alpha}T$ is still yet to be seen.

Since the optimisation problem of the original reversible pebble game is known to be PSPACE-complete [Chan et al. 2015], we expect that our pebble game for Qurts is also PSPACE-complete. If that is indeed the case, there could be a reduction from the pebble game to the problem of Quantified Boolean Formulas (QBF), which is a well-known PSPACE-complete problem. Thus, instead of using specific heuristic algorithms made for uncomputation, the optimisation power of a generic QBF solver could be used as a backend for the Qurts compiler to optimise the uncomputation.

One advanced way to optimise quantum uncomputation that has gained attention is so-called spooky pebble games [Gidney 2019]. This is a pebble game that models a way to uncompute quantum states using measurement, and is thus not reversible. Spooky pebble games have been shown to model more efficient computation than the original reversible pebble game in some situations [Kornerup et al. 2024; Quist and Laarman 2023, 2024]. Since our uncomputation semantics uses reversible pebble games as a backend, it can be easily extended spooky pebble games. It would be interesting to see much this can optimise Qurts performance.

The uncomputation semantics exhibited strong parallelism in executing two branches of `qif`. Both branches can not only share a reference $\&T$, they can also write to the same location. In classical programming, this normally produces race conditions or deadlock. However, Qurts' uncomputation semantics are thread safe, as we have shown, because both branches are controlled by a qubit. A promising direction for future research is to analyze this behavior through the lens of ownership.

Acknowledgments

The authors would like to thank the anonymous referees for their valuable comments and helpful suggestions. We also sincerely thank Ryo Wakizaka, Yusuke Matsushita, Sam Lindley, and other colleagues in Tokyo, Chiba, Kyoto, and Edinburgh for their valuable discussions.

References

- Thorsten Altenkirch and Jonathan Grattage. 2005. A Functional Quantum Programming Language. In *20th IEEE Symposium on Logic in Computer Science (LICS 2005), 26-29 June 2005, Chicago, IL, USA, Proceedings*. IEEE Computer Society, 249–258. <https://doi.org/10.1109/LICS.2005.1>
- Matthew Amy, Martin Roetteler, and Krysta M. Svore. 2017. Verified Compilation of Space-Efficient Reversible Circuits. In *Computer Aided Verification - 29th International Conference, CAV 2017, Heidelberg, Germany, July 24-28, 2017, Proceedings, Part II (Lecture Notes in Computer Science, Vol. 10427)*, Rupak Majumdar and Viktor Kuncak (Eds.). Springer, 3–21. https://doi.org/10.1007/978-3-319-63390-9_1
- Charles H. Bennett. 1989. Time/Space Trade-Offs for Reversible Computation. *SIAM J. Comput.* 18, 4 (1989), 766–776. <https://doi.org/10.1137/0218053>
- Debjoyoti Bhattacharjee, Mathias Soeken, Srijit Dutta, Anupam Chattopadhyay, and Giovanni De Micheli. 2019. Reversible Pebble Games for Reducing Qubits in Hierarchical Quantum Circuit Synthesis. In *2019 IEEE 49th International Symposium on Multiple-Valued Logic (ISMVL), Fredericton, NB, Canada, May 21-23, 2019*. IEEE, 102–107. <https://doi.org/10.1109/ISMVL.2019.00026>
- Benjamin Bichsel, Maximilian Baader, Timon Gehr, and Martin T. Vechev. 2020. Silq: a high-level quantum language with safe uncomputation and intuitive semantics. In *Proceedings of the 41st ACM SIGPLAN International Conference on Programming Language Design and Implementation, PLDI 2020, London, UK, June 15-20, 2020*, Alastair F. Donaldson and Emina Torlak (Eds.). ACM, 286–300. <https://doi.org/10.1145/3385412.3386007>
- Siu Man Chan, Massimo Lauria, Jakob Nordström, and Marc Vinyals. 2015. Hardness of Approximation in PSPACE and Separation Results for Pebble Games. In *IEEE 56th Annual Symposium on Foundations of Computer Science, FOCS 2015, Berkeley, CA, USA, 17-20 October, 2015*, Venkatesan Guruswami (Ed.). IEEE Computer Society, 466–485. <https://doi.org/10.1109/FOCS.2015.36>
- Yongshan Ding, Xin-Chuan Wu, Adam Holmes, Ash Wiseth, Diana Franklin, Margaret Martonosi, and Frederic T. Chong. 2020. SQUARE: Strategic Quantum Ancilla Reuse for Modular Quantum Programs via Cost-Effective Uncomputation. In *47th ACM/IEEE Annual International Symposium on Computer Architecture, ISCA 2020, Virtual Event / Valencia, Spain, May 30 - June 3, 2020*. IEEE, 570–583. <https://doi.org/10.1109/ISCA45697.2020.00054>
- Peng Fu, Kohei Kishida, Neil J. Ross, and Peter Selinger. 2020. A Tutorial Introduction to Quantum Circuit Programming in Dependently Typed Proto-Quipper. In *Reversible Computation - 12th International Conference, RC 2020, Oslo, Norway, July 9-10, 2020, Proceedings (Lecture Notes in Computer Science, Vol. 12227)*, Ivan Lanese and Mariusz Rawski (Eds.). Springer, 153–168. https://doi.org/10.1007/978-3-030-52482-1_9
- Craig Gidney. 2019. Spooky pebble games and irreversible uncomputation. <https://algassert.com/post/1905>
- Lov K. Grover. 1996. A fast quantum mechanical algorithm for database search. In *Proceedings of the Twenty-Eighth Annual ACM Symposium on Theory of Computing (Philadelphia, Pennsylvania, USA) (STOC '96)*. Association for Computing Machinery, New York, NY, USA, 212–219. <https://doi.org/10.1145/237814.237866>
- Kengo Hirata and Chris Heunen. 2024. Qurts: Automatic Quantum Uncomputation by Affine Types with Lifetime. <https://doi.org/10.48550/arXiv.2411.10835> arXiv:2411.10835 [cs.PL]
- Ali JavadiAbhari, Shruti Patil, Daniel Kudrow, Jeff Heckey, Alexey Lvov, Frederic T. Chong, and Margaret Martonosi. 2014. ScaffCC: a framework for compilation and analysis of quantum computing programs. In *Computing Frontiers Conference, CF'14, Cagliari, Italy - May 20 - 22, 2014*, Pedro Trancoso, Diana Franklin, and Sally A. McKee (Eds.). ACM, 1:1–1:10. <https://doi.org/10.1145/2597917.2597939>
- Ali JavadiAbhari, Shruti Patil, Daniel Kudrow, Jeff Heckey, Alexey Lvov, Frederic T. Chong, and Margaret Martonosi. 2015. ScaffCC: Scalable compilation and analysis of quantum programs. *Parallel Comput.* 45 (2015), 2–17. <https://doi.org/10.1016/j.parco.2014.12.001>
- Niels Kornerup, Jonathan Sadun, and David Soloveichik. 2024. Tight Bounds on the Spooky Pebble Game: Recycling Qubits with Measurements. <https://doi.org/10.48550/arXiv.2110.08973> arXiv:2110.08973 [quant-ph]
- Arun Kumar Pati and Samuel L. Braunstein. 2000. Impossibility of deleting an unknown quantum state. *Nature* 404, 6774 (2000), 164–165. <https://doi.org/10.1038/404130b0>
- Marco Lewis and Timon Gehr. 2021. Silq Issue 28: Quantum redefinition through quantum control behaviour inconsistent depending on scope. <https://github.com/eth-sri/silq/issues/28>. <https://github.com/eth-sri/silq/issues/28> GitHub Issue.
- Nicholas D. Matsakis and Felix S. Klock. 2014. The rust language. In *Proceedings of the 2014 ACM SIGAda Annual Conference on High Integrity Language Technology (Portland, Oregon, USA) (HILT '14)*. Association for Computing Machinery, New

- York, NY, USA, 2 pages. <https://doi.org/10.1145/2663171.2663188>
- Yusuke Matsushita, Takeshi Tsukada, and Naoki Kobayashi. 2020. RustHorn: CHC-Based Verification for Rust Programs. In *Programming Languages and Systems - 29th European Symposium on Programming, ESOP 2020, Held as Part of the European Joint Conferences on Theory and Practice of Software, ETAPS 2020, Dublin, Ireland, April 25-30, 2020, Proceedings (Lecture Notes in Computer Science, Vol. 12075)*, Peter Müller (Ed.). Springer, 484–514. https://doi.org/10.1007/978-3-030-44914-8_18
- Giulia Meuli, Mathias Soeken, Martin Roetteler, Nikolaj S. Björner, and Giovanni De Micheli. 2019. Reversible Pebbling Game for Quantum Memory Management. In *Design, Automation & Test in Europe Conference & Exhibition, DATE 2019, Florence, Italy, March 25-29, 2019*, Jürgen Teich and Franco Fummi (Eds.). IEEE, 288–291. <https://doi.org/10.23919/DATE.2019.8715092>
- Michael A. Nielsen and Isaac L. Chuang. 2010. *Quantum Computation and Quantum Information: 10th Anniversary Edition*. Cambridge University Press. <https://doi.org/10.1017/CBO9780511976667>
- Anouk Paradis, Benjamin Bichsel, Samuel Steffen, and Martin T. Vechev. 2021. Unqomp: synthesizing uncomputation in Quantum circuits. In *PLDI '21: 42nd ACM SIGPLAN International Conference on Programming Language Design and Implementation, Virtual Event, Canada, June 20-25, 2021*, Stephen N. Freund and Eran Yahav (Eds.). ACM, 222–236. <https://doi.org/10.1145/3453483.3454040>
- Anouk Paradis, Benjamin Bichsel, and Martin T. Vechev. 2024. Reqomp: Space-constrained Uncomputation for Quantum Circuits. *Quantum* 8 (2024), 1258. <https://doi.org/10.22331/Q-2024-02-19-1258>
- Vaughan Pratt. 1992. Linear Logic for Generalized Quantum Mechanics. In *Workshop on Physics and Computation*. 166–180. <https://doi.org/10.1109/PHYCMP.1992.615518>
- Arend-Jan Quist and Alfons Laarman. 2023. Optimizing Quantum Space Using Spooky Pebble Games. In *Reversible Computation - 15th International Conference, RC 2023, Giessen, Germany, July 18-19, 2023, Proceedings (Lecture Notes in Computer Science, Vol. 13960)*, Martin Kutrib and Uwe Meyer (Eds.). Springer, 134–149. https://doi.org/10.1007/978-3-031-38100-3_10
- Arend-Jan Quist and Alfons Laarman. 2024. Trade-offs between classical and quantum space using spooky pebbling. <https://doi.org/10.48550/arXiv.2401.10579> arXiv:2401.10579 [quant-ph]
- Robert Rand, Jennifer Paykin, Dong-Ho Lee, and Steve Zdancewic. 2018. ReQWIRE: Reasoning about Reversible Quantum Circuits. In *Proceedings 15th International Conference on Quantum Physics and Logic, QPL 2018, Halifax, Canada, 3-7th June 2018 (EPTCS, Vol. 287)*, Peter Selinger and Giulio Chiribella (Eds.). 299–312. <https://doi.org/10.4204/EPTCS.287.17>
- Rust Community. 2024a. *References and Borrowing – The Rust Programming Language*. <https://doc.rust-lang.org/book/ch04-02-references-and-borrowing.html>
- Rust Community. 2024b. *The Rust Programming Language*. <http://rust-lang.org/>
- Rust Community. 2024c. *Validating References with Lifetimes – The Rust Programming Language*. <https://doc.rust-lang.org/book/ch10-03-lifetime-syntax.html>
- Rust Community. 2024d. *What Is Ownership? – The Rust Programming Language*. <https://doc.rust-lang.org/book/ch04-01-what-is-ownership.html>
- Amr Sabry, Benoît Valiron, and Juliana Kaizer Vizzotto. 2018. From Symmetric Pattern-Matching to Quantum Control. In *Foundations of Software Science and Computation Structures - 21st International Conference, FOSSACS 2018, Held as Part of the European Joint Conferences on Theory and Practice of Software, ETAPS 2018, Thessaloniki, Greece, April 14-20, 2018, Proceedings (Lecture Notes in Computer Science, Vol. 10803)*, Christel Baier and Ugo Dal Lago (Eds.). Springer, 348–364. https://doi.org/10.1007/978-3-319-89366-2_19
- Raphael Seidel, Nikolay Tcholchev, Sebastian Bock, and Manfred Hauswirth. 2023. Uncomputation in the Qrisp High-Level Quantum Programming Framework. In *Reversible Computation - 15th International Conference, RC 2023, Giessen, Germany, July 18-19, 2023, Proceedings (Lecture Notes in Computer Science, Vol. 13960)*, Martin Kutrib and Uwe Meyer (Eds.). Springer, 150–165. https://doi.org/10.1007/978-3-031-38100-3_11
- Peter Selinger and Benoît Valiron. 2006. A lambda calculus for quantum computation with classical control. *Math. Struct. Comput. Sci.* 16, 3 (2006), 527–552. <https://doi.org/10.1017/S0960129506005238>
- Hristo Venev, Timon Gehr, Dimitar Dimitrov, and Martin Vechev. 2024. Modular Synthesis of Efficient Quantum Uncomputation. *Proceedings of the ACM on Programming Languages* 8, OOPSLA2, Article 345 (2024). <https://doi.org/10.1145/3689785>
- Philip Wadler. 1990. Linear Types can Change the World!. In *Programming concepts and methods: Proceedings of the IFIP Working Group 2.2, 2.3 Working Conference on Programming Concepts and Methods, Sea of Galilee, Israel, 2-5 April, 1990*, Manfred Broy and Cliff B. Jones (Eds.). North-Holland, 561.
- William K. Wootters and Wojciech H. Zurek. 1982. A single quantum cannot be cloned. *Nature* 299, 5886 (1982), 802–803. <https://doi.org/10.1038/299802a0>
- Noson S. Yanofsky and Mirco Mannucci. 2008. *Quantum Computing for Computer Scientists*. Cambridge University Press. <https://doi.org/10.1017/CBO9780511813887>
- Mingsheng Ying. 2016. *Foundations of Quantum Programming* (1st ed.). Morgan Kaufmann Publishers Inc., San Francisco, CA, USA. <https://doi.org/10.1016/C2014-0-02660-3>

Charles Yuan, Christopher McNally, and Michael Carbin. 2022. Twist: sound reasoning for purity and entanglement in Quantum programs. *Proc. ACM Program. Lang.* 6, POPL (2022), 1–32. <https://doi.org/10.1145/3498691>

Received 2024-07-10; accepted 2024-11-07